Nguyên Lí HDH

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

Mục lục

- Chương 1: Tổng quan: Bài 1: Khái niệm về hệ điều hành
- Chương 1: Tổng quan: Bài 2: Phân loại hệ điều hành
- Chương 2: Tiểu trình và tiến trình: Bài 3: Nhu cầu xử lí đồng thời
- Chương 2: Tiểu trình và tiến trình: Bài 4: Khái niệm tiến trình và mô hình đa tiến trình
- Chương 2: Tiểu trình và tiến trình: Bài 5: Khái niệm tiến trình và mô hình tiểu tiến trình
- Chương 3: Lập trình và tiến trình: Bài 6: Tổ chức quản lí tiến trình
- Chương 3: Lập trình và tiến trình: Bài 7: Lập trình tiến trình
- Chương 4: Truyền thông và đồng bộ tiến trình: Bài 8: Liên lạc tiến trình
- Chương 4: Truyền thông và đồng bộ tiến trình: Bài 9: Các cơ chế thông tin liên lạc
- Chương 4: Truyền thông và đồng bộ tiến trình: Bài 10: Nhu cầu đồng bộ hóa
- Chương 4: Truyền thông và đồng bộ tiến trình: Bài 11: Các giải pháp đồng bộ hóa

CHƯƠNG 1 · BÀI 1

1.1. Khái niệm về hệ điều hành

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Hiểu hệ điều hành là gì và vị trí của nó trong hệ thống máy tính.
- Nêu được bốn thành phần chính của một hệ thống máy tính.
- Giải thích vai trò quản lý và phân phối tài nguyên của hệ điều hành.
- Nắm được hai mục tiêu cơ bản của hệ điều hành.

Hệ điều hành là một chương trình hay một hệ chương trình hoạt động giữa người sử dụng (user) và phần cứng của máy tính. Mục tiêu của hệ điều hành là cung cấp một môi trường để người sử dụng có thể thi hành các chương trình. Nó làm cho máy tính dễ sử dụng hơn, thuận lợi hơn và hiệu quả hơn. Hệ điều hành là một phần quan trọng của hầu hết các hệ thống máy tính. Một hệ thống máy tính thường được chia làm bốn phần chính: phần cứng, hệ điều hành, các chương trình ứng dụng và người sử dụng.

Phần cứng bao gồm CPU, bộ nhớ, các thiết bị nhập xuất, đây là những tài nguyên của máy tính. Chương trình ứng dụng như các chương trình dịch, hệ thống cơ sở dữ liệu, các trò chơi, và các chương trình thương mại. Các chương trình này sử dụng tài nguyên của máy tính để giải quyết các yêu cầu của người sử dụng. Hệ điều hành điều khiển và phối hợp việc sử dụng phần cứng cho những ứng dụng khác nhau của nhiều người sử dụng khác nhau. Hệ điều hành cung cấp một môi trường mà các chương trình có thể làm việc hữu hiệu trên đó.

Hệ điều hành có thể được coi như là bộ phân phối tài nguyên của máy tính. Nhiều tài nguyên của máy tính như thời gian sử dụng CPU, vùng bộ nhớ, vùng lưu trữ tập tin, thiết bị nhập xuất v.v… được các chương trình yêu cầu để giải quyết vấn đề. Hệ điều hành hoạt động như một bộ quản lý các tài nguyên và phân phối chúng cho các chương trình và người sử dụng khi cần thiết. Do có rất nhiều yêu cầu, hệ điều hành phải giải quyết vấn đề tranh chấp và phải quyết định cấp phát tài nguyên cho những yêu cầu theo thứ tự nào để hoạt động của máy tính là hiệu quả nhất. Một hệ điều hành cũng có thể được coi như là một chương trình kiểm soát việc sử dụng máy tính, đặc biệt là các thiết bị nhập xuất.

Tuy nhiên, nhìn chung chưa có định nghĩa nào là hoàn hảo về hệ điều hành. Hệ điều hành tồn tại để giải quyết các vấn đề sử dụng hệ thống máy tính. Mục tiêu cơ bản của nó là giúp cho việc thi hành các chương trình dễ dàng hơn. Mục tiêu thứ hai là hỗ trợ cho các thao tác trên hệ thống máy tính hiệu quả hơn. Mục tiêu này đặc biệt quan trọng trong những hệ thống nhiều người dùng và trong những hệ thống lớn (phần cứng + quy mô sử

dụng). Tuy nhiên hai mục tiêu này cũng có phần tương phản, vì vậy lý thuyết về hệ điều hành tập trung vào việc tối ưu hóa việc sử dụng tài nguyên của máy tính.

Câu hỏi ôn tập

1. Hệ điều hành là gì? Nó nằm ở vị trí nào giữa người sử dụng và phần cứng?
2. Kể tên bốn thành phần chính của một hệ thống máy tính.
3. Vì sao nói hệ điều hành là “bộ phân phối tài nguyên” của máy tính?
4. Nêu hai mục tiêu cơ bản của hệ điều hành. Vì sao hai mục tiêu này có phần tương phản?

Bài 1: Khái niệm về hệ điều hành — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "Máy mở nhiều app thì lag, liên quan gì hệ điều hành?" Ứng dụng cần CPU, bộ nhớ và thiết bị I/O. Hệ điều hành quản lý và phân phối các tài nguyên đó; nhiều công việc có thể cạnh tranh tài nguyên hoặc chờ I/O. Chỉ từ chữ "lag" chưa thể kết luận thiếu RAM hay ứng dụng nào lỗi.

Tự luyện: ghi nhận lúc lag, số ứng dụng, CPU, RAM và hoạt động đĩa từ công cụ hệ thống. **Hướng giải:** đối chiếu thời điểm thay vì nhìn một ảnh chụp bất kỳ. Không coi CPU thấp là chắc chắn máy không có vấn đề: công việc có thể đang chờ tài nguyên khác.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 1 · BÀI 2

1.2. Phân loại hệ điều hành

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Phân biệt được sáu loại hệ thống: xử lý theo lô, lô đa chương, chia sẻ thời gian, song song, phân tán và thời gian thực.
- Hiểu các khái niệm: bộ giám sát thường trực, xử lý off-line, spooling, đa chương, đa nhiệm.
- Phân biệt đa xử lý đối xứng và bất đối xứng; thời gian thực cứng và mềm.

1.2.1 Hệ thống xử lý theo lô

– **Bộ giám sát thường trực:** Khi một công việc chấm dứt, hệ thống sẽ thực hiện công việc kế tiếp mà không cần sự can thiệp của người lập trình, do đó thời gian thực hiện sẽ mau hơn. Một chương trình, còn gọi là bộ giám sát thường trực, được thiết kế để giám sát việc thực hiện dãy các công việc một cách tự động, chương trình này luôn luôn thường trú trong bộ nhớ chính. Hệ điều hành theo lô thực hiện các công việc lần lượt theo những chỉ thị định trước.

– **CPU và thao tác nhập xuất:** CPU thường hay nhàn rỗi do tốc độ làm việc của các thiết bị nhập xuất (thường là thiết bị cơ) chậm hơn rất nhiều lần so với các thiết bị điện tử. Cho dù là một CPU chậm nhất, nó cũng nhanh hơn rất nhiều lần so với thiết bị nhập xuất. Do đó phải có các phương pháp để đồng bộ hóa việc hoạt động của CPU và thao tác nhập xuất.

– **Xử lý off-line:** Xử lý off-line là thay vì CPU phải đọc trực tiếp từ thiết bị nhập và xuất ra thiết bị xuất, hệ thống dùng một bộ lưu trữ trung gian. CPU chỉ thao tác với bộ phận này. Việc đọc hay xuất đều đến và từ bộ lưu trữ trung gian.

– **Spooling:** Spool (Simultaneous Peripheral Operation On-Line – thao tác ngoại vi đồng thời trực tuyến) là đồng bộ hóa các thao tác bên ngoài on-line. Cơ chế này cho phép xử lý của CPU là on-line, sử dụng đĩa để lưu các dữ liệu nhập cũng như xuất.

1.2.2 Hệ thống xử lý theo lô đa chương

Khi có nhiều công việc cùng truy xuất lên thiết bị, vấn đề lập lịch cho các công việc là cần thiết. Khía cạnh quan trọng nhất trong việc lập lịch là khả năng đa chương. Đa chương (multiprogram) gia tăng khai thác CPU bằng cách tổ chức các công việc sao cho CPU luôn luôn phải trong tình trạng làm việc.

Ý tưởng: hệ điều hành lưu giữ một phần của các công việc ở nơi lưu trữ trong bộ nhớ. CPU sẽ lần lượt thực hiện các phần công việc này. Khi đang thực hiện, nếu có yêu cầu truy xuất thiết bị thì CPU không nghỉ mà thực hiện tiếp công việc thứ hai… Với hệ đa chương, hệ điều hành ra quyết định cho người sử dụng, vì vậy hệ điều hành đa chương rất tinh vi. Hệ phải xử lý các vấn đề lập lịch cho công việc, lập lịch cho bộ nhớ và cho cả CPU nữa.

1.2.3 Hệ thống chia sẻ thời gian

Hệ thống chia sẻ thời gian là một mở rộng logic của hệ đa chương. Hệ thống này còn được gọi là hệ thống đa nhiệm (multitasking). Nhiều công việc cùng được thực hiện thông qua cơ chế chuyển đổi của CPU như hệ đa chương nhưng thời gian mỗi lần chuyển đổi diễn ra rất nhanh. Hệ thống chia sẻ được phát triển để cung cấp việc sử dụng bên trong của một máy tính có giá trị hơn. Hệ điều hành chia sẻ thời gian dùng lập lịch CPU và đa chương để cung cấp cho mỗi người sử dụng một phần nhỏ trong máy tính chia sẻ. Một chương trình khi thi hành được gọi là một tiến trình. Trong quá trình thi hành của một tiến trình, nó phải thực hiện các thao tác nhập xuất và trong khoảng thời gian đó CPU sẽ thi hành một tiến trình khác. Hệ điều hành chia sẻ cho phép nhiều người sử dụng chia sẻ máy tính một cách đồng bộ; do thời gian chuyển đổi nhanh nên họ có cảm giác là các tiến trình đang được thi hành cùng lúc.

Hệ điều hành chia sẻ phức tạp hơn hệ điều hành đa chương. Nó phải có các chức năng: quản trị và bảo vệ bộ nhớ, sử dụng bộ nhớ ảo. Nó cũng cung cấp hệ thống tập tin truy xuất on-line… Hệ điều hành chia sẻ là kiểu của các hệ điều hành hiện đại ngày nay.

1.2.4 Hệ thống song song

Ngoài các hệ thống chỉ có một bộ xử lý còn có các hệ thống có nhiều bộ xử lý cùng chia sẻ hệ thống đường truyền dữ liệu, đồng hồ, bộ nhớ và các thiết bị ngoại vi. Các bộ xử lý này liên lạc bên trong với nhau. Có nhiều nguyên nhân xây dựng dạng hệ thống này. Với sự gia tăng số lượng bộ xử lý, công việc được thực hiện nhanh chóng hơn. Nhưng không phải theo đúng tỉ lệ thời gian, nghĩa là có n bộ xử lý không có nghĩa là sẽ thực hiện nhanh hơn n lần.

Hệ thống với máy nhiều bộ xử lý sẽ tối ưu hơn hệ thống có nhiều máy có một bộ xử lý vì các bộ xử lý chia sẻ các thiết bị ngoại vi, hệ thống lưu trữ, nguồn… và rất thuận tiện cho nhiều chương trình cùng làm việc trên cùng một tập hợp dữ liệu.

Một lý do nữa là độ tin cậy. Các chức năng được xử lý trên nhiều bộ xử lý và sự hỏng hóc của một bộ xử lý sẽ không ảnh hưởng đến toàn bộ hệ thống.

Hệ thống đa xử lý thông thường sử dụng cách **đa xử lý đối xứng**, trong cách này mỗi bộ xử lý chạy với một bản sao của hệ điều hành, những bản sao này liên lạc với nhau khi cần thiết. Một số hệ thống sử dụng **đa xử lý bất đối xứng**, trong đó mỗi bộ xử lý được giao

một công việc riêng biệt. Một bộ xử lý chính kiểm soát toàn bộ hệ thống, các bộ xử lý khác thực hiện theo lệnh của bộ xử lý chính hoặc theo những chỉ thị đã được định nghĩa trước. Mô hình này theo dạng quan hệ chủ – tớ. Bộ xử lý chính sẽ lập lịch cho các bộ xử lý khác.

Một ví dụ về hệ thống xử lý đối xứng là phiên bản Encore của UNIX cho máy tính Multimax. Hệ thống này có hàng tá bộ xử lý. Ưu điểm của nó là nhiều tiến trình có thể thực hiện cùng lúc. Một hệ thống đa xử lý cho phép nhiều công việc và tài nguyên được chia sẻ tự động trong những bộ xử lý khác nhau.

Hệ thống đa xử lý không đồng bộ thường xuất hiện trong những hệ thống lớn, trong đó hầu hết thời gian hoạt động đều dành cho xử lý nhập xuất.

1.2.5 Hệ thống phân tán

Hệ thống này cũng tương tự như hệ thống chia sẻ thời gian nhưng các bộ xử lý không chia sẻ bộ nhớ và đồng hồ, thay vào đó mỗi bộ xử lý có bộ nhớ cục bộ riêng. Các bộ xử lý thông tin với nhau thông qua các đường truyền thông như những bus tốc độ cao hay đường dây điện thoại. Các bộ xử lý trong hệ phân tán thường khác nhau về kích thước và chức năng. Nó có thể bao gồm máy vi tính, trạm làm việc, máy mini, và những hệ thống máy lớn. Các bộ xử lý thường được tham khảo với nhiều tên khác nhau như site, node, computer v.v… tùy thuộc vào trạng thái làm việc của chúng.

Các nguyên nhân phải xây dựng hệ thống phân tán là:

- **Chia sẻ tài nguyên:** Một người sử dụng A có thể sử dụng máy in laser của người sử dụng B và người sử dụng B có thể truy xuất những tập tin của A. Tổng quát, chia sẻ tài nguyên trong hệ thống phân tán cung cấp một cơ chế để chia sẻ tập tin ở vị trí xa, xử lý thông tin trong một cơ sở dữ liệu phân tán, in ấn tại một vị trí xa, sử dụng những thiết bị ở xa để thực hiện các thao tác.
- **Tăng tốc độ tính toán:** Một thao tác tính toán được chia làm nhiều phần nhỏ cùng thực hiện một lúc. Hệ thống phân tán cho phép phân chia việc tính toán trên nhiều vị trí khác nhau để tính toán song song.
- **An toàn:** Nếu một vị trí trong hệ thống phân tán bị hỏng, các vị trí khác vẫn tiếp tục làm việc.
- **Thông tin liên lạc với nhau:** Có nhiều lúc, chương trình cần chuyển đổi dữ liệu từ vị trí này sang vị trí khác. Ví dụ trong hệ thống Windows, thường có sự chia sẻ và chuyển dữ liệu giữa các cửa sổ. Khi các vị trí được nối kết với nhau trong một hệ thống mạng, việc trao đổi dữ liệu diễn ra rất dễ. Người sử dụng có thể chuyển tập tin hay các e-mail cho nhau từ cùng vị trí hay những vị trí khác.

1.2.6 Hệ thống xử lý thời gian thực

Hệ thống xử lý thời gian thực được sử dụng khi có những đòi hỏi khắt khe về thời gian trên các thao tác của bộ xử lý hoặc dòng dữ liệu, nó thường được dùng điều khiển các thiết bị trong các ứng dụng tận hiến (dedicated). Máy tính phân tích dữ liệu và có thể chỉnh các điều khiển giải quyết cho dữ liệu nhập. Một hệ điều hành xử lý thời gian thực phải được định nghĩa tốt, thời gian xử lý nhanh. Hệ thống phải cho kết quả chính xác trong khoảng thời gian bị thúc ép nhanh nhất. Có hai hệ thống xử lý thời gian thực là hệ thống thời gian thực cứng và hệ thống thời gian thực mềm.

Hệ thống thời gian thực cứng là công việc được hoàn tất đúng lúc. Lúc đó dữ liệu thường được lưu trong bộ nhớ ngắn hạn hay trong ROM. Việc xử lý theo thời gian thực sẽ xung đột với tất cả hệ thống liệt kê ở trên.

Dạng thứ hai là **hệ thống thời gian thực mềm**, mỗi công việc có một độ ưu tiên riêng và sẽ được thi hành theo độ ưu tiên đó. Có một số lĩnh vực áp dụng hữu hiệu phương pháp này là multimedia hay thực tại ảo.

Câu hỏi ôn tập

1. Bộ giám sát thường trực là gì? Nó giúp gì cho hệ thống xử lý theo lô?
2. Phân biệt xử lý off-line và spooling.
3. Trình bày ý tưởng của hệ thống xử lý theo lô đa chương.
4. Hệ thống chia sẻ thời gian khác hệ đa chương ở điểm nào?
5. Vì sao có n bộ xử lý không có nghĩa là hệ thống chạy nhanh hơn n lần?
6. So sánh đa xử lý đối xứng và đa xử lý bất đối xứng.
7. Hệ thống phân tán khác hệ thống song song ở điểm nào? Nêu bốn lý do xây dựng hệ thống phân tán.
8. Phân biệt hệ thống thời gian thực cứng và thời gian thực mềm, cho ví dụ ứng dụng.

Bài 2: Phân loại hệ điều hành — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhầm đa nhiệm, nhiều người dùng và thời gian thực là cùng một loại. Các tiêu chí phân loại có thể giao nhau: một hệ thống có thể đa nhiệm và hỗ trợ nhiều người dùng. Hệ thời gian thực quan tâm đáp ứng ràng buộc thời hạn, không chỉ "chạy nhanh".

Tự luyện: so máy cá nhân, máy chủ và bộ điều khiển có thời hạn phản ứng. **Hướng giải:** nêu yêu cầu và hậu quả khi trễ, không chỉ gán nhãn theo tên thương mại. Phân biệt hệ đa xử lý có nhiều bộ xử lý với cảm giác nhiều tác vụ cùng chạy trên một CPU.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 2 · BÀI 1

2.1. Nhu cầu xử lý đồng thời

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Hiểu vì sao hệ điều hành hiện đại cần hỗ trợ xử lý đồng thời.
- Giải thích được cách đa nhiệm giúp tăng hiệu suất sử dụng CPU.
- Nhận biết các bài toán có thể tăng tốc nhờ xử lý song song.

Có 2 động lực chính khiến cho các hệ điều hành hiện đại thường hỗ trợ môi trường đa nhiệm (multitask) trong đó chấp nhận nhiều tác vụ thực hiện đồng thời trên cùng một máy tính:

2.1.1 Tăng hiệu suất sử dụng CPU

Phần lớn các tác vụ (job) khi thi hành đều trải qua nhiều chu kỳ xử lý (sử dụng CPU) và chu kỳ nhập xuất (sử dụng các thiết bị nhập xuất) xen kẽ như sau:

CPU	IO	CPU	IO	CPU

Nếu chỉ có 1 tiến trình duy nhất trong hệ thống, thì vào các chu kỳ IO của tác vụ, CPU sẽ hoàn toàn nhàn rỗi. Ý tưởng tăng cường số lượng tác vụ trong hệ thống là để tận dụng CPU: nếu tác vụ 1 xử lý IO, thì có thể sử dụng CPU để thực hiện tác vụ 2...

CPU	IO	CPU	IO	CPU

CPU	IO	CPU	IO	CPU

2.1.2 Tăng tốc độ xử lý

Một số bài toán có bản chất xử lý song song nếu được xây dựng thành nhiều module hoạt động đồng thời thì sẽ tiết kiệm được thời gian xử lý.

Ví dụ: Xét bài toán tính giá trị biểu thức kq = a*b + c*d. Nếu tiến hành tính đồng thời (a*b) và (c*d) thì thời gian xử lý sẽ ngắn hơn là thực hiện tuần tự. Trong các trường hợp đó, cần có một mô hình xử lý đồng hành thích hợp. Trên máy tính có cấu hình nhiều CPU, hỗ trợ xử lý song song (multiprocessing) thật sự, điều này sẽ giúp tăng hiệu quả thi hành của hệt thống đáng kể.

Câu hỏi ôn tập

1. Nêu hai động lực chính khiến hệ điều hành hỗ trợ môi trường đa nhiệm.
2. Vì sao khi chỉ có một tiến trình, CPU bị nhàn rỗi trong các chu kỳ nhập xuất?
3. Lấy một ví dụ bài toán có thể tăng tốc độ xử lý nhờ tính toán đồng thời.

Bài 3: Nhu cầu xử lí đồng thời — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: tải file trong khi nghe nhạc. Đồng thời là quản lý nhiều công việc có thời gian sống chồng nhau; song song là thực thi thực sự cùng lúc ở mức tài nguyên xử lý. Một lõi có thể xen kẽ công việc để tạo tính đáp ứng.

Tự luyện: vẽ timeline tác vụ A dùng CPU rồi chờ I/O, tác vụ B cần CPU. **Hướng giải:** cho B sử dụng CPU trong lúc A chờ có thể giảm thời gian nhàn rỗi; không kết luận tổng thời gian luôn giảm một nửa. Đo thời gian chờ và thời gian hoàn thành riêng.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 2 · BÀI 2

2.2. Khái niệm tiến trình và mô hình đa tiến trình

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

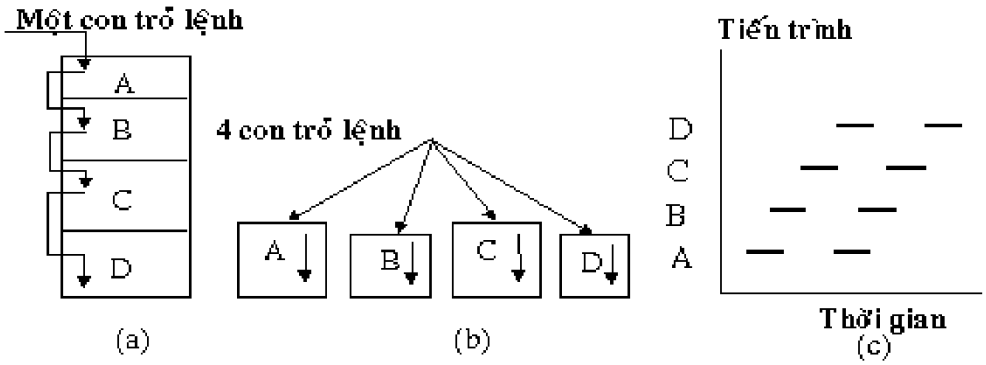
Mục tiêu bài học

- Nắm được khái niệm tiến trình và mô hình đa tiến trình.
- Phân biệt chương trình (thực thể thụ động) và tiến trình (thực thể hoạt động).
- Hiểu vai trò của bộ điều phối (scheduler).

Để hỗ trợ sự đa chương, máy tính phải có khả năng thực hiện nhiều tác vụ đồng thời. Nhưng việc điều khiển nhiều hoạt động song song ở cấp độ phần cứng là rất khó khăn. Vì thế các nhà thiết kế hệ điều hành đề xuất một mô hình song song gia lặp bằng cách chuyển đổi bộ xử lý qua lại giữa các chương trình để duy trì hoạt động của nhiều chương trình cùng lúc, điều này tạo cảm giác có nhiều hoạt động được thực hiện đồng thời.

Trong mô hình này, tất cả các phần mềm trong hệ thống được tổ chức thành một số những tiến trình (process). Tiến trình là một chương trình đang xử lý, sỡ hữu một con trỏ lệnh, tập các thanh ghi và các biến. Để hoàn thành tác vụ của mình, một tiến trình có thể cần đến một số tài nguyên – như CPU, bộ nhớ chính, các tập tin và thiết bị nhập/xuất. Cần phân biệt hai khái niệm chương trình và tiến trình. Một chương trình là một thực thể thụ động, chứa đựng các chỉ thị điều khiển máy tính để tiến hành một tác vụ nào đó ; khi cho thực hiện các chỉ thị này, chương trình chuyển thành tiến trình, tiến trình là một thực thể hoạt động, với con trỏ lệnh xác định chỉ thị kế tiếp sẽ thi hành, kèm theo tập các tài nguyên phục vụ cho hoạt động của tiến trình. Về mặt ý niệm, có thể xem như mỗi tiến trình sỡ hữu một bộ xử lý ảo cho riêng nó, nhưng trong thực tế, chỉ có một bộ xử lý thật sự được chuyển đổi qua lại giữa các tiến trình. Sự chuyển đổi nhanh chóng này được gọi là sự đa chương (multiprogramming) .

Hệ điều hành chịu trách nhiệm sử dụng một thuật toán điều phối để quyết định thời điểm cần dừng hoạt động của tiến trình đang xử lý để phục vụ một tiến trình khác, và lựa chọn tiến trình tiếp theo sẽ được phục vụ. Bộ phận thực hiện chức năng này của hệ điều hành được gọi là bộ điều phối (scheduler).



Hình 2.1 (a) Đa chương với 4 chương trình
(b) Mô hình khái niệm với 4 chương trình độc lập
(c) Tại một thời điểm chỉ có một chương trình hoạt động

Câu hỏi ôn tập

1. Tiến trình là gì? Một tiến trình sở hữu những thành phần nào?
2. Phân biệt chương trình và tiến trình.
3. Bộ điều phối (scheduler) của hệ điều hành làm nhiệm vụ gì?

Bài 4: Khái niệm tiến trình và mô hình đa tiến trình — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "Chương trình khác tiến trình thế nào?" Chương trình là mã và dữ liệu ở dạng thụ động; tiến trình là một lần thực thi có trạng thái, ngữ cảnh CPU và tài nguyên. Mở cùng ứng dụng hai lần có thể tạo nhiều tiến trình; chi tiết còn tùy thiết kế ứng dụng.

Ví dụ: quán cà phê có nhiều phiếu gọi món; mỗi phiếu là một công việc đang được xử lý, còn công thức là chỉ dẫn. Ví dụ chỉ giúp hình dung, không đồng nhất nhân viên với mọi thành phần của hệ điều hành.

Tự luyện: vẽ ready → running → blocked khi đọc file. **Hướng giải:** ready đang đợi CPU, blocked đang đợi sự kiện/tài nguyên; chưa được chạy không có nghĩa đã kết thúc.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 2 · BÀI 3

2.3. Khái niệm tiểu trình và mô hình đa tiểu trình

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Hiểu khái niệm tiểu trình (thread) và mô hình đa tiểu trình.
- Nắm được thông tin nào được chia sẻ giữa các tiểu trình và thông tin nào là riêng.
- Phân biệt kernel thread và user thread.

Trong hầu hết các hệ điều hành, mỗi tiến trình có một không gian địa chỉ và chỉ có một dòng xử lý. Tuy nhiên, có nhiều tình huống người sử dụng mong muốn có nhiều dòng xử lý cùng chia sẻ một không gian địa chỉ, và các dòng xử lý này hoạt động song song tương tự như các tiến trình phân biệt (ngoại trừ việc chia sẻ không gian địa chỉ). Ví dụ: Một server quản lý tập tin thỉnh thoảng phải tự khóa để chờ các thao tác truy xuất đĩa hoàn tất.Nếu server có nhiều dòng xử lý, hệ thống có thể xử lý các yêu cầu mới trong khi một dòng xử lý bị khoá. Như vậy việc thực hiện chương trình sẽ có hiệu quả hơn.

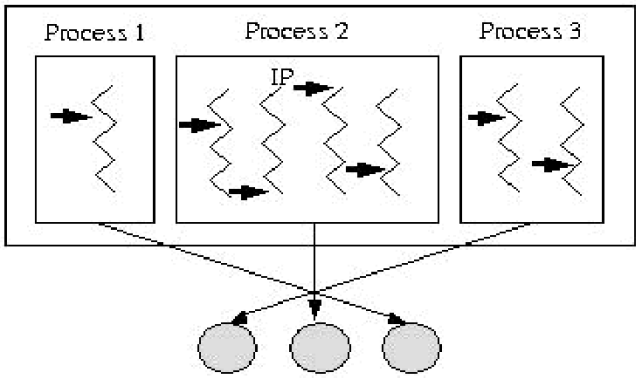
Điều này không thể đạt được bằng cách tạo hai tiến trình server riêng biệt vì cần phải chia sẻ cùng một vùng đệm, do vậy bắt buộc phải chia sẻ không gian địa chỉ. Chính vì các tình huống tương tự, người ta cần có một cơ chế xử lý mới cho phép có nhiều dòng xử lý trong cùng một tiến trình. Ngày nay đã có nhiều hệ điều hành cung cấp một cơ chế như thế và gọi là tiểu trình (threads).

2.3.1 Nguyên lý chung

Một tiểu trình là một đơn vị xử lý cơ bản trong hệ thống. Mỗi tiểu trình xử lý tuần tự đoạn code của nó, sỡ hữu một con trỏ lệnh, tập các thanh ghi và một vùng nhớ stack riêng. Các tiểu trình chia sẻ CPU với nhau giống như cách chia sẻ giữa các tiến trình: một tiểu trình xử lý trong khi các tiểu trình khác chờ đến lượt. Một tiểu trình cũng có thể tạo lập các tiến trình con, và nhận các trạng thái khác nhau như một tiến trình thật sự. Một tiến trình có thể sỡ hữu nhiều tiểu trình.

Các tiến trình tạo thành những thực thể độc lập. Mỗi tiến trình có một tập tài nguyên và một môi trường riêng (một con trỏ lệnh, một Stack, các thanh ghi và không gian địa chỉ). Các tiến trình hoàn toàn độc lập với nhau, chỉ có thể liên lạc thông qua các cơ chế thông tin giữa các tiến trình mà hệ điều hành cung cấp. Ngược lại, các tiểu trình trong cùng một tiến trình lại chia sẻ một không gian địa chỉ chung, điều này có nghĩa là các tiểu trình có thể chia sẻ các biến toàn cục của tiến trình. Một tiểu trình có thể truy xuất đến cả các

stack của những tiểu trình khác trong cùng tiến trình. Cấu trúc này không đề nghị một cơ chế bảo vệ nào, và điều này cũng không thật cần thiết vì các tiểu trình trong cùng một tiến trình thuộc về cùng một sỡ hữu chủ đã tạo ra chúng trong ý định cho phép chúng hợp tác với nhau.



Các tiểu trình trong cùng một tiểu trình

2.3.2 Phân bổ thông tin lưu trữ

Tiến trình
Không gian địa chỉ Tài nguyên toàn cục Các thông tin thống kê

Tiểu trình
Con trỏ lệnh + các thanh ghi Stack Tài nguyên cục bộ

Cấu trúc miêu tả tiểu trình và tiến trình

2.3.3 Kernel thread và userthread

Khái niệm tiểu trình có thể được cài đặt trong Kernel của hệ điều hành, khi đó đơn vị cơ sở sử dụng CPU để xử lý tiểu trình, hệ điều hành sẽ phân phối CPU cho các tiểu trình trong hệ thống. Tuy nhiên đối với một số hệ điều hành, khái niệm tiểu trình chỉ được hỗ trợ như một đối tượng người dùng, các thao tác tiểu trình được cung cấp kèm theo do một bộ thư viện xử lý trong chế độ người dùng không đặc quyền (user mode). Lúc này hệ điều hành sẽ chỉ biết đến khái niệm tiểu trình, do vậy cần có cơ chế để liên kết các tiểu trình cùng một tiến trình với tiến trình cha trong Kernel, đối tượng này đôi lúc được gọi là LWP (light weight process).

Câu hỏi ôn tập

1. Tiểu trình là gì? Vì sao cần có nhiều dòng xử lý trong cùng một tiến trình?
2. Các tiểu trình trong cùng một tiến trình chia sẻ những gì, và mỗi tiểu trình có những gì riêng?
3. So sánh kernel thread và user thread: ưu, nhược điểm của từng loại.

Bài 5: Khái niệm tiến trình và mô hình tiểu tiến trình — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: thêm thread nhưng không nhanh hơn. Các thread trong một tiến trình chia sẻ một số tài nguyên và không gian địa chỉ, nhưng mỗi thread có luồng điều khiển, ngữ cảnh và stack riêng. Công việc phải có phần có thể thực hiện đồng thời và không bị nút thắt khác lấn át.

Tự luyện: phân biệt dữ liệu dùng chung và biến cục bộ trong mô hình hai thread. **Hướng giải:** dữ liệu chia sẻ cần cơ chế truy cập đúng; "cùng tiến trình" không làm phép cập nhật nhiều bước tự trở nên nguyên tử. Tránh suy từ số thread sang tốc độ nếu chưa đo.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 3 · BÀI 1

3.1. Tổ chức quản lý tiến trình

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

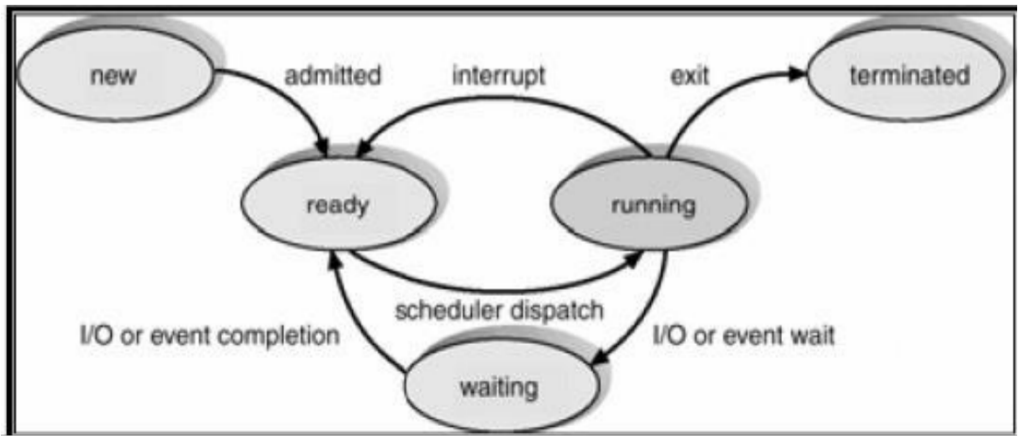
- Nắm được các trạng thái của tiến trình và sự chuyển đổi giữa chúng.
- Hiểu cấu trúc khối quản lý tiến trình (PCB).
- Biết các thao tác tạo lập, kết thúc tiến trình và cách cấp phát tài nguyên cho tiến trình.

3.1.1 Các trạng thái của tiến trình

Trạng thái của tiến trình tại một thời điểm được xác định bởi hoạt động hiện thời của tiến trình tại thời điểm đó. Trong quá trình sống, một tiến trình thay đổi trạng thái do nhiều nguyên nhân như: phải chờ một sự kiện nào đó xảy ra, hay đợi một thao tác nhập/xuất hoàn tất, buộc phải dừng hoạt động do đã hết thời gian xử lý …

Tại một thời điểm, một tiến trình có thể nhận trong một các trạng thái sau đây:

- Mới tạo: tiến trình đang được tạo lập.
- Running: các chỉ thị của tiến trình đang được xử lý.
- Waiting: tiến trình chờ được cấp phát một tài nguyên, hay chờ một sự kiện xảy ra.
- Ready: tiến trình chờ được cấp phát CPU để xử lý.
- Kết thúc: tiến trình hoàn tất xử lý.



***Hình 3.1.1-1**. Sơ đồ chuyển trạng thái giữa các tiến trình*

Tại một thời điểm, chỉ có một tiến trình có thể nhận trạng thái running trên một bộ xử lý bất kỳ. Trong khi đó, nhiều tiến trình có thể ở trạng thái blocked hay ready. Các cung chuyển tiếp trong sơ đồ trạng thái biểu diễn sáu sự chuyển trạng thái có thể xảy ra trong các điều kiện sau:

- Tiến trình mới tạo được đưa vào hệ thống (bộ nhớ trong)
- Bộ điều phối cấp phát cho tiến trình một khoảng thời gian sử dụng CPU
- Tiến trình kết thúc
- Tiến trình yêu cầu một tài nguyên nhưng chưa được đáp ứng vì tài nguyên chưa sẵn sàng để cấp phát tại thời điểm đó; hoặc tiến trình phải chờ một sự kiện hay thao tác nhập/xuất.
- Bộ điều phối chọn một tiến trình khác để cho xử lý .
- Tài nguyên mà tiến trình yêu cầu trở nên sẵn sàng để cấp phát; hay sự kiện hoặc thao tác nhập/xuất tiến trình đang đợi hoàn tất.

3.1.2 Chế độ xử lý của tiến trình

Để đảm bảo hệ thống hoạt động đúng đắn, hệ điều hành cần phải được bảo vệ khỏi sự xâm phạm của các tiến trình. Bản thân các tiến trình và dữ liệu cũng cần được bảo vệ để tránh các ảnh hưởng sai lạc lẫn nhau. Một cách tiếp cận để giải quyết vấn đề là phân biệt hai chế độ xử lý cho các tiến trình: chế độ không đặc quyền và chế độ đặc quyền nhờ vào sự trợ giúp của cơ chế phần cứng. Tập lệnh của CPU được phân chia thành các lệnh đặc quyền và lệnh không đặc quyền. Cơ chế phần cứng chỉ cho phép các lệnh đặc quyền được thực hiện trong chế độ đặc quyền. Thông thường chỉ có hệ điều hành hoạt động trong chế độ đặc quyền, các tiến trình của người dùng hoạt động trong chế độ không đặc quyền, không thực hiện được các lệnh đặc quyền có nguy cơ ảnh hưởng đến hệ thống. Như vậy hệ điều hành được bảo vệ. Khi một tiến trình người dùng gọi đến một lời gọi hệ thống, tiến trình của hệ điều hành xử lý lời gọi này sẽ hoạt động trong chế độ đặc quyền, sau khi hoàn tất thì trả quyền điều khiển về cho tiến trình người dùng trong chế độ không đặc quyền.

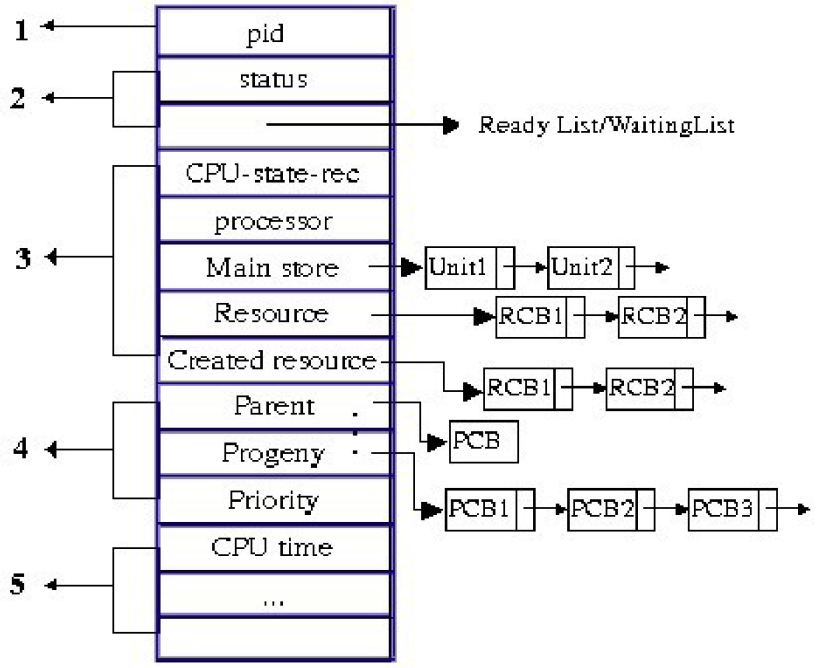


***Hình 3.1.2-1**. Hai chế độ xử lý*

3.1.3 Cấu trúc dữ liệu khối quản lý tiến trình

Hệ điều hành quản lý các tiến trình trong hệ thống thông qua khối quản lý tiến trình (process control block -PCB). PCB là một vùng nhớ lưu trữ các thông tin mô tả cho tiến trình, với các thành phần chủ yếu bao gồm:

- Định danh của tiến trình (1): giúp phân biệt các tiến trình
- Trạng thái tiến trình (2): xác định hoạt động hiện hành của tiến trình.
- Ngữ cảnh của tiến trình (3): mô tả các tài nguyên tiến trình đang trong quá trình, hoặc để phục vụ cho hoạt động hiện tại, hoặc để làm cơ sở phục hồi hoạt động cho tiến trình, bao gồm các thông tin về:
  - Trạng thái CPU: bao gồm nội dung các thanh ghi, quan trọng nhất là con trỏ lệnh IP lưu trữ địa chỉ câu lệnh kế tiếp tiến trình sẽ xử lý. Các thông tin này cần được lưu trữ khi xảy ra một ngắt, nhằm có thể cho phép phục hồi hoạt động của tiến trình đúng như trước khi bị ngắt.
  - Bộ xử lý: dùng cho máy có cấu hình nhiều CPU, xác định số hiệu CPU mà tiến trình đang sử dụng.
  - Bộ nhớ chính: danh sách các khối nhớ được cấp cho tiến trình.
  - Tài nguyên sử dụng: danh sách các tài mguyên hệ thống mà tiến trình đang sử dụng.
  - Tài nguyên tạo lập: danh sách các tài nguyên được tiến trình tạo lập.
- Thông tin giao tiếp (4): phản ánh các thông tin về quan hệ của tiến trình với các tiến trình khác trong hệ thống:
  - Tiến trình cha: của một tiến trình là tiến trình tạo lập ra tiến trình này
  - Tiến trình con: của 1 tiến trình là các tiến trình do tiến trình này tạo lập .
  - Độ ưu tiên: giúp bộ điều phối có thông tin để lựa chọn tiến trình được cấp CPU.
- Thông tin thống kê (5): đây là những thông tin thống kê về hoạt động của tiến trình, như thời gian đã sử dụng CPU, thời gian chờ. Các thông tin này có thể có ích cho công việc đánh giá tình hình hệ thống và dự đoán các tình huống tương lai.



***Hình 3.1.3-1**. Khối điều khiển tiến trình*

3.1.4 Thao tác trên tiến trình

Hệ điều hành cung cấp các thao tác chủ yếu sau đây trên một tiến trình:

- Tạo lập tiến trình (create)
- Kết thúc tiến trình (destroy)
- Tạm dừng tiến trình (suspend)
- Tái kích hoạt tiến trình (resume)
- Thay đổi độ ưu tiên tiến trình

3.1.4.1 Tạo lập tiến trình

Trong quá trình xử lý, một tiến trình có thể tạo lập nhiều tiến trình mới bằng cách sử dụng một lời gọi hệ thống tương ứng. Tiến trình gọi lời gọi hệ thống để tạo tiến trình mới sẽ được gọi là tiến trình cha, tiến trình được tạo gọi là tiến trình con. Mỗi tiến trình con đến lượt nó lại có thể tạo các tiến trình mới… quá trình này tiếp tục sẽ tạo ra một cây tiến trình.

Các công việc hệ điều hành cần thực hiện khi tạo lập tiến trình bao gồm

- Định danh cho tiến trình mới phát sinh
- Đưa tiến trình vào danh sách quản lý của hệ thống
- Xác định độ ưu tiên cho tiến trình
- Tạo PCB cho tiến trình

– Cấp phát các tài nguyên ban đầu cho tiến trình

Khi một tiến trình tạo lập một tiến trình con, tiến trình con có thể sẽ được hệ điều hành trực tiếp cấp phát tài nguyên hoặc được tiến trình cha cho thừa hưởng một số tài nguyên ban đầu.

Khi một tiến trình tạo tiến trình mới, tiến trình ban đầu có thể xử lý theo một trong hai khả năng sau:

– Tiến trình cha tiếp tục xử lý đồng hành với tiến trình con.
– Tiến trình cha chờ đến khi một tiến trình con nào đó, hoặc tất cả các tiến trình con kết thúc xử lý.

Các hệ điều hành khác nhau có thể chọn lựa các cài đặt khác nhau để thực hiện thao tác tạo lập một tiến trình.

3.1.4.2 Kết thúc tiến trình

Một tiến trình kết thúc xử lý khi nó hoàn tất chỉ thị cuối cùng và sử dụng một lời gọi hệ thống để yêu cầu hệ điều hành hủy bỏ nó (giải phóng CPU). Đôi khi một tiến trình có thể yêu cầu hệ điều hành kết thúc xử lý của một tiến trình khác. Khi một tiến trình kết thúc, hệ điều hành thực hiện các công việc:

– Thu hồi các tài nguyên hệ thống đã cấp phát cho tiến trình
– Hủy tiến trình khỏi tất cả các danh sách quản lý của hệ thống
– Hủy bỏ PCB của tiến trình

Hầu hết các hệ điều hành không cho phép các tiến trình con tiếp tục tồn tại nếu tiến trình cha đã kết thúc. Trong những hệ thống như thế, hệ điều hành sẽ tự động phát sinh một loạt các thao tác kết thúc tiến trình con.

3.1.5 Cấp phát tài nguyên cho tiến trình

Khi có nhiều người sử dụng đồng thời làm việc trong hệ thống, hệ điều hành cần phải cấp phát các tài nguyên theo yêu cầu cho mỗi người sử dụng. Do tài nguyên hệ thống thường rất giới hạn và có khi không thể chia sẻ, nên hiếm khi tất cả các yêu cầu tài nguyên đồng thời đều được thỏa mãn. Vì thế cần phải nghiên cứu một phương pháp để chia sẻ một số tài nguyên hữu hạn giữa nhiều tiến trình người dùng đồng thời. Hệ điều hành quản lý nhiều loại tài nguyên khác nhau (CPU, bộ nhớ chính, các thiết bị ngoại vi …), với mỗi loại cần có một cơ chế cấp phát và các chiến lược cấp phát hiệu qủa. Mỗi tài nguyên được biểu diễn thông qua một cấu trúc dữ liệu, khác nhau về chi tiết cho từng loại tài nguyên, nhưng cơ bản chứa đựng các thông tin sau :

– Định danh tài nguyên

- Trạng thái tài nguyên: đây là các thông tin mô tả chi tiết trạng thái tài nguyên: phần nào của tài nguyên đã cấp phát cho tiến trình, phần nào còn có thể sử dụng?
- Hàng đợi trên một tài nguyên: danh sách các tiến trình đang chờ được cấp phát tài nguyên tương ứng.
- Bộ cấp phát: là đoạn code đảm nhiệm việc cấp phát một tài nguyên đặc thù.

Một số tài nguyên đòi hỏi các giải thuật đặc biệt (như CPU, bộ nhớ chính, hệ thống tập tin), trong khi những tài nguyên khác (như các thiết bị nhập/xuất) có thể cần các giải thuật cấp phát và giải phóng tổng quát hơn.



Hình 3.1.5-1. *Khối quản lý tài nguyên Các mục tiêu của kỹ thuật cấp phát :*

- Bảo đảm một số lượng hợp lệ các tiến trình truy xuất đồng thời đến các tài nguyên không chia sẻ được.
- Cấp phát tài nguyên cho tiến trình có yêu cầu trong một khoảng thời gian trì hoãn có thể chấp nhận được.
- Tối ưu hóa sự sử dụng tài nguyên.

Để có thể thõa mãn các mục tiêu kể trên, cần phải giải quyết các vấn đề nảy sinh khi có nhiều tiến trình đồng thời yêu cầu một tài nguyên không thể chia sẻ.

Câu hỏi ôn tập

1. Vẽ sơ đồ và giải thích các trạng thái của một tiến trình.
2. Khối quản lý tiến trình (PCB) gồm những thông tin gì?
3. Trình bày các bước hệ điều hành thực hiện khi tạo lập và khi kết thúc một tiến trình.
4. Hệ điều hành cấp phát tài nguyên cho tiến trình như thế nào?

Bài 6: Tổ chức quản lí tiến trình — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: một tác vụ ngắn phải đợi sau tác vụ dài. Mô phỏng FCFS với thời lượng 6, 2, 1 cùng đến lúc 0: thứ tự đó cho thời gian chờ 0, 6, 8; trung bình 14/3. Thay chính sách có thể cải thiện một chỉ số nhưng ảnh hưởng công bằng hoặc chi phí chuyển ngữ cảnh.

Tự luyện: đổi thứ tự rồi mô phỏng Round Robin quantum 2. **Hướng giải:** ghi thời điểm đến, CPU burst, giả định chi phí và timeline trước khi tính. Không so hai kết quả dựa trên hai bộ giả định khác nhau hoặc bỏ qua starvation khi bàn về ưu tiên.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 3 · BÀI 2

3.2. Lập lịch tiến trình

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Hiểu mục tiêu của lập lịch và các đặc điểm của tiến trình ảnh hưởng đến lập lịch.
- Phân biệt điều phối độc quyền và không độc quyền; các danh sách và cấp độ lập lịch.
- Vận dụng được các thuật toán FIFO, Round Robin, độ ưu tiên, SJF, đa mức, xổ số để tính thời gian chờ.

Trong môi trường đa chương, có thể xảy ra tình huống nhiều tiến trình đồng thời sẵn sàng để xử lý. Mục tiêu của các hệ phân chia thời gian (time-sharing) là chuyển đổi CPU qua lại giữa các tiến trình một cách thường xuyên để nhiều người sử dụng có thể tương tác cùng lúc với từng chương trình trong quá trình xử lý.

Để thực hiện được mục tiêu này, hệ điều hành phải lựa chọn tiến trình được xử lý tiếp theo. Bộ điều phối sẽ sử dụng một giải thuật điều phối thích hợp để thực hiện nhiệm vụ này. Một thành phần khác của hệ điều hành cũng tiềm ẩn trong công tác điều phối là bộ phân phối (dispatcher). Bộ phân phối sẽ chịu trách nhiệm chuyển đổi ngữ cảnh và trao CPU cho tiến trình được chọn bởi bộ điều phối để xử lý.

3.2.1 Giới thiệu

3.2.1.1 Mục tiêu lập lịch

Bộ điều phối không cung cấp cơ chế, mà đưa ra các quyết định. Các hệ điều hành xây dựng nhiều chiến lược khác nhau để thực hiện việc điều phối, nhưng tựu chung cần đạt được các mục tiêu sau:

- Sự công bằng (Fairness): Các tiến trình chia sẻ CPU một cách công bằng, không có tiến trình nào phải chờ đợi vô hạn để được cấp phát CPU.
- Tính hiệu qủa (Efficiency): Hệ thống phải tận dụng được CPU 100% thời gian.
- Thời gian đáp ứng hợp lý (Response time): Cực tiểu hoá thời gian hồi đáp cho các tương tác của người sử dụng.
- Thời gian lưu lại trong hệ thống (Turnaround Time): Cực tiểu hóa thời gian hoàn tất các tác vụ xử lý theo lô.
- Thông lượng tối đa (Throughput): Cực đại hóa số công việc được xử lý trong một đơn vị thời gian.

Tuy nhiên thường không thể thỏa mãn tất cả các mục tiêu kể trên vì bản thân chúng có sự mâu thuẫn với nhau mà chỉ có thể dung hòa chúng ở mức độ nào đó.

3.2.1.2 Các đặc điểm của tiến trình

Điều phối hoạt động của các tiến trình là một vấn đề rất phức tạp, đòi hỏi hệ điều hành khi giải quyết phải xem xét nhiều yếu tố khác nhau để có thể đạt được những mục tiêu đề ra. Một số đặc tính của tiến trình cần được quan tâm như tiêu chuẩn điều phối:

- Tính hướng xuất/nhập của tiến trình (I/O-boundedness): Khi một tiến trình nhận được CPU, chủ yếu nó chỉ sử dụng CPU đến khi phát sinh một yêu cầu nhập xuất? Hoạt động của các tiến trình như thế thường bao gồm nhiều lượt sử dụng CPU, mỗi lượt trong một thời gian khá ngắn.
- Tính hướng xử lý của tiến trình (CPU-boundedness): Khi một tiến trình nhận được CPU, nó có khuynh hướng sử dụng CPU đến khi hết thời gian dành cho nó? Hoạt động của các tiến trình như thế thường bao gồm một số ít lượt sử dụng CPU, nhưng mỗi lượt trong một thời gian đủ dài.
- Tiến trình tương tác hay xử lý theo lô: Người sử dụng theo kiểu tương tác thường yêu cầu được hồi đáp tức thời đối với các yêu cầu của họ, trong khi các tiến trình của tác vụ được xử lý theo lô nói chung có thể trì hoãn trong một thời gian chấp nhận được.
- Độ ưu tiên của tiến trình: Các tiến trình có thể được phân cấp theo một số tiêu chuẩn đánh giá nào đó, một cách hợp lý, các tiến trình quan trọng hơn (có độ ưu tiên cao hơn) cần được ưu tiên hơn.
- Thời gian đã sử dụng CPU của tiến trình: Một số quan điểm ưu tiên chọn những tiến trình đã sử dụng CPU nhiều thời gian nhất vì hy vọng chúng sẽ cần ít thời gian nhất để hoàn tất và rời khỏi hệ thống.

Tuy nhiên cũng có quan điểm cho rằng các tiến trình nhận được CPU trong ít thời gian là những tiến trình đã phải chờ lâu nhất, do vậy ưu tiên chọn chúng.

- Thời gian còn lại tiến trình cần để hoàn tất: Có thể giảm thiểu thời gian chờ đợi trung bình của các tiến trình bằng cách cho các tiến trình cần ít thời gian nhất để hoàn tất được thực hiện trước. Tuy nhiên đáng tiếc là rất hiếm khi biết được tiến trình cần bao nhiêu thời gian nữa để kết thúc xử lý.

3.2.1.3 Điều phối không độc quyền và điều phối độc quyền (preemptive/nopreemptive)

Thuật toán điều phối cần xem xét và quyết định thời điểm chuyển đổi CPU giữa các tiến trình. Hệ điều hành có thể thực hiện cơ chế điều phối theo nguyên lý độc quyền hoặc không độc quyền.

- Điều phối độc quyền: Nguyên lý điều phối độc quyền cho phép một tiến trình khi nhận được CPU sẽ có quyền độc chiếm CPU đến khi hoàn tất xử lý hoặc tự nguyện giải phóng CPU. Khi đó quyết định điều phối CPU sẽ xảy ra trong các tình huống sau:
  - Khi tiến trình chuyển từ trạng thái đang xử lý (running) sang trạng thái bị khóa blocked (ví dụ chờ một thao tác nhập xuất hay chờ một tiến trình con kết thúc…).
  - Khi tiến trình kết thúc.

Các giải thuật độc quyền thường đơn giản và dễ cài đặt. Tuy nhiên chúng thường không thích hợp với các hệ thống tổng quát nhiều người dùng, vì nếu cho phép một tiến trình có quyền xử lý bao lâu tùy ý, có nghĩa là tiến trình này có thể giữ CPU một thời gian không xác định, có thể ngăn cản những tiến trình còn lại trong hệ thống có một cơ hội để xử lý.

- Điều phối không độc quyền: Ngược với nguyên lý độc quyền, điều phối theo nguyên lý không độc quyền cho phép tạm dừng hoạt động của một tiến trình đang sẵn sàng xử lý. Khi một tiến trình nhận được CPU, nó vẫn được sử dụng CPU đến khi hoàn tất hoặc tự nguyện giải phóng CPU, nhưng khi có một tiến trình khác có độ ưu tiên có thể dành quyền sử dụng CPU của tiến trình ban đầu. Như vậy là tiến trình có thể bị tạm dừng hoạt động bất cứ lúc nào mà không được báo trước, để tiến trình khác xử lý. Các quyết định điều phối xảy ra khi :
  - Khi tiến trình chuyển từ trạng thái đang xử lý (running) sang trạng thái bị khóa blocked (ví dụ chờ một thao tác nhập xuất hay chờ một tiến trình con kết thúc…).
  - Khi tiến trình chuyển từ trạng thái đang xử lý (running) sang trạng thái ready (ví dụ xảy ra một ngắt).
  - Khi tiến trình chuyển từ trạng thái chờ (blocked) sang trạng thái ready (ví dụ một thao tác nhập/xuất hoàn tất).
  - Khi tiến trình kết thúc.

Các thuật toán điều phối theo nguyên tắc không độc quyền ngăn cản được tình trạng một tiến trình độc chiếm CPU, nhưng việc tạm dừng một tiến trình có thể dẫn đến các mâu thuẫn trong truy xuất, đòi hỏi phải sử dụng một phương pháp đồng bộ hóa thích hợp để giải quyết. Trong các hệ thống sử dụng nguyên lý điều phối độc quyền có thể xảy ra tình trạng các tác vụ cần thời gian xử lý ngắn phải chờ tác vụ xử lý với thời gian rất dài hoàn tất!

Nguyên lý điều phối độc quyền thường chỉ thích hợp với các hệ xử lý theo lô.

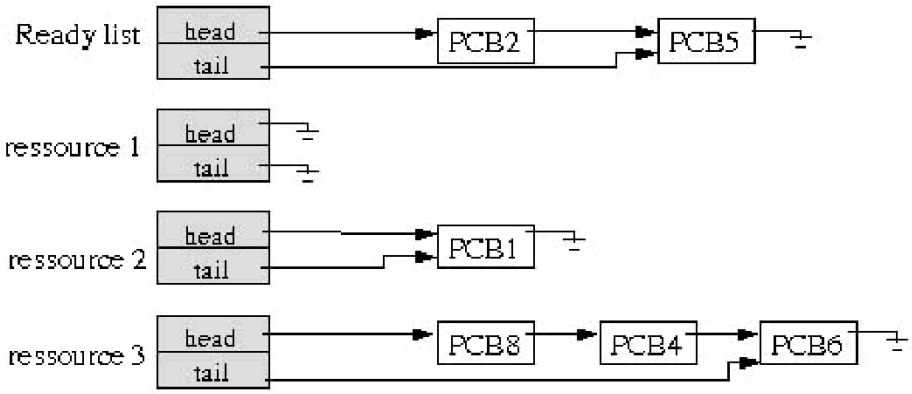
Đối với các hệ thống tương tác(time sharing), các hệ thời gian thực (real time), cần phải sử dụng nguyên lý điều phối không độc quyền để các tiến trình quan trọng có cơ hội hồi đáp kịp thời. Tuy nhiên thực hiện điều phối theo nguyên lý không độc quyền đòi hỏi những cơ chế phức tạp trong việc phân định độ ưu tiên, và phát sinh thêm chi phí khi chuyển đổi CPU qua lại giữa các tiến trình.

3.2.2 Tổ chức lập lịch

3.2.2.1 Các danh sách sử dụng trong quá trình lập lịch

Hệ điều hành sử dụng hai loại danh sách để thực hiện điều phối các tiến trình là danh sách sẵn sàng (ready list) và danh sách chờ đợi(waiting list). Khi một tiến trình bắt đầu đi vào hệ thống, nó được chèn vào danh sách các tác vụ (job list). Danh sách này bao gồm tất cả các tiến trình của hệ thống. Nhưng chỉ các tiến trình đang thường trú trong bộ nhớ chính và ở trạng thái sẵn sàng tiếp nhận CPU để hoạt động mới được đưa vào danh sách sẵn sàng. Bộ điều phối sẽ chọn một tiến trình trong danh sách sẵn sàng và cấp CPU cho tiến trình đó. Tiến trình được cấp CPU sẽ thực hiện xử lý, và có thể chuyển sang trạng thái chờ khi xảy ra các sự kiện như đợi một thao tác nhập/xuất hoàn tất, yêu cầu tài nguyên chưa được thỏa mãn, được yêu cầu tạm dừng ... Khi đó tiến trình sẽ được chuyển sang một danh sách chờ đợi.

Hệ điều hành chỉ sử dụng một danh sách sẵn sàng cho toàn hệ thống, nhưng mỗi một tài nguyên (thiết bị ngoại vi) có một danh sách chờ đợi riêng bao gồm các tiến trình đang chờ được cấp phát tài nguyên đó.

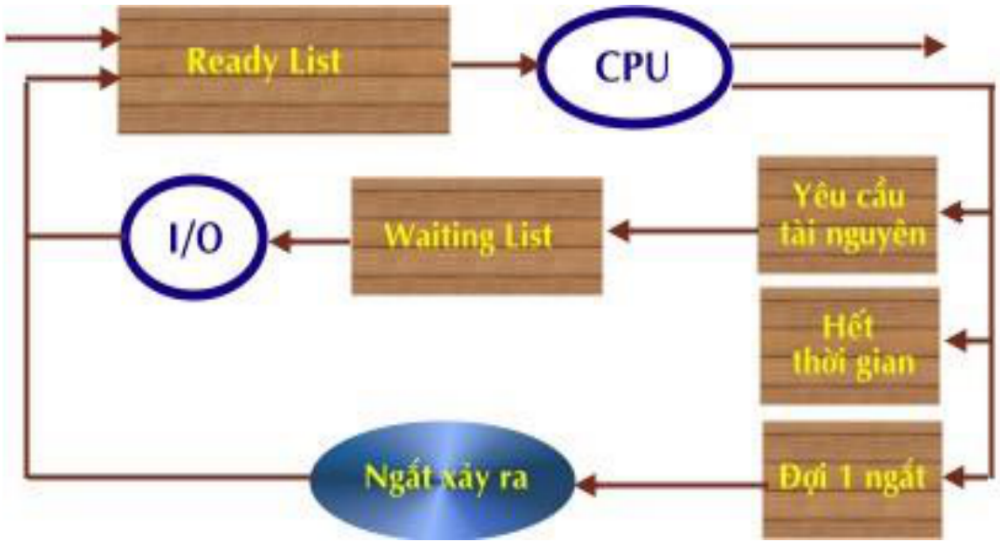


***Hình 3.2.2.1-1**. Các danh sách điều phối*

Quá trình xử lý của một tiến trình trải qua những chu kỳ chuyển đổi qua lại giữa danh sách sẵn sàng và danh sách chờ đợi. Sơ đồ dưới đây mô tả sự điều phối các tiến trình dựa trên các danh sách của hệ thống. Thoạt đầu tiến trình mới được đặt trong danh sách các tiến

trình sẵn sàng (ready list), nó sẽ đợi trong danh sách này cho đến khi được chọn để cấp phát CPU và bắt đầu xử lý. Sau đó có thể xảy ra một trong các tình huống sau:

- Tiến trình phát sinh một yêu cầu một tài nguyên mà hệ thống chưa thể đáp ứng, khi đó tiến trình sẽ được chuyển sang danh sách các tiến trình đang chờ tài nguyên tương ứng.
- Tiến trình có thể bị bắt buộc tạm dừng xử lý do một ngắt xảy ra, khi đó tiến trình được đưa trở lại vào danh sách sẵn sàng để chờ được cấp CPU cho lượt tiếp theo.



***Hình 3.2.2.1-2**. Sơ đồ chuyển đổi giữa các danh sách điều phối*

Trong trường hợp đầu tiên, tiến trình cuối cùng sẽ chuyển từ trạng thái blocked sang trạng thái ready và lại được đưa trở vào danh sách sẵn sàng. Tiến trình lặp lại chu kỳ này cho đến khi hoàn tất tác vụ thì được hệ thống hủy bỏ khỏi mọi danh sách điều phối.

3.2.2.2 Các cấp độ lập lịch

Thực ra công việc điều phối được hệ điều hành thực hiện ở hai mức độ: điều phối tác vụ (job scheduling) và điều phối tiến trình (process scheduling).

a) Lập lịch tác vụ

Quyết định lựa chọn tác vụ nào được đưa vào hệ thống, và nạp những tiến trình của tác vụ đó vào bộ nhớ chính để thực hiện. Chức năng điều phối tác vụ quyết định mức độ đa chương của hệ thống (số lượng tiến trình trong bộ nhớ chính). Khi hệ thống tạo lập một tiến trình, hay có một tiến trình kết thúc xử lý thì chức năng điều phối tác vụ mới được kích hoạt. Vì mức độ đa chương tương đối ổn định nên chức năng điều phối tác vụ có tần suất hoạt động thấp. Để hệ thống hoạt động tốt, bộ điều phối tác vụ cần biệt tính chất của tiến trình là hướng nhập xuất (I/O bounded) hay hướng xử lý (CPU bounded). Một tiến trình được gọi là hướng nhập xuất nếu nó chủ yếu nó chỉ sử dụng CPU để thực hiện các thao tác

nhập xuất. Ngược lại một tiến trình được gọi là hướng xử lý nếu nó chủ yếu nó chỉ sử dụng CPU để thực hiện các thao tác tính toán. Để cân bằng hoạt động của CPU và các thiết bị ngoại vi, bộ điều phối tác vụ nên lựa chọn các tiến trình để nạp vào bộ nhớ sao cho hệ thống là sự pha trộn hợp lý giữa các tiến trình hướng nhập xuất và các tiến trình hướng xử lý

b) Lập lịch tiến trình

Chọn một tiến trình ở trạng thái sẵn sàng (đã được nạp vào bộ nhớ chính, và có đủ tài nguyên để hoạt động) và cấp phát CPU cho tiến trình đó thực hiện. Bộ điều phối tiến trình có tần suất hoạt động cao, sau mỗi lần xảy ra ngắt (do đồng hồ báo giờ, do các thiết bị ngoại vi...), thường là 1 lần trong khoảng 100ms. Do vậy để nâng cao hiệu suất của hệ thống, cần phải tăng tốc độ xử lý của bộ điều phối tiến trình. Chức năng điều phối tiến trình là một trong chức năng cơ bản, quan trọng nhất của hệ điều hành.

Trong nhiều hệ điều hành, có thể không có bộ điều phối tác vụ hoặc tách biệt rất ít đối với bộ điều phối tiến trình. Một vài hệ điều hành lại đưa ra một cấp độ điều phối trung gian kết hợp cả hai cấp độ điều phối tác vụ và tiến trình.

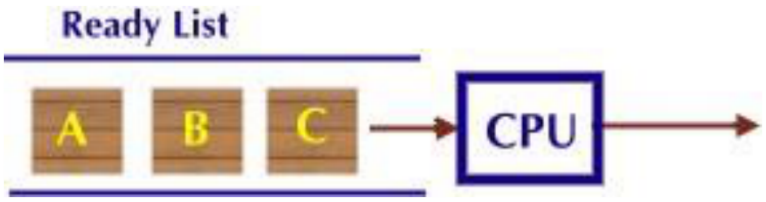


***Hình 3.2.2.2-1**. Cấp độ điều phối trung gian*

3.2.3 Các thuật toán lập lịch

3.2.3.1 Chiến lược FIFO

- **Nguyên tắc**: CPU được cấp phát cho tiến trình đầu tiên trong danh sách sẵn sàng có yêu cầu, là tiến trình được đưa vào hệ thống sớm nhất. Đây là thuật toán điều phối theo nguyên tắc độc quyền. Một khi CPU được cấp phát cho tiến trình, CPU chỉ được tiến trình tự nguyện giải phóng khi kết thúc xử lý hay khi có một yêu cầu nhập/xuất.



***Hình 3.2.3.1-1**. Điều phối FIFO Ví dụ :*

Tiến trình	Thời điểm vào RL	Thời gian xử lý
P1	0	24
P2	1	3
P3	2	3

Thứ tự cấp phát CPU cho các tiến trình là :

P1	P2	P3
0	24	27 30

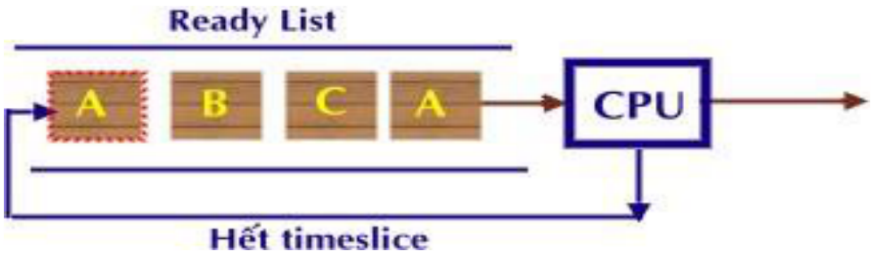
Thời gian chờ đợi được xử lý là 0 đối với P1, (24 -1) với P2 và (24+3-2) với P3. Thời gian chờ trung bình là (0+23+25)/3 = 16 milisecondes.

Giải thuật này đặc biệt không phù hợp với các hệ phân chia thời gian, trong các hệ này, cần cho phép mỗi tiến trình được cấp phát CPU đều đặn trong từng khoảng thời gian.

3.2.3.2 Lập lịch xoay vòng (Round Robin)

– **Nguyên tắc**: Danh sách sẵn sàng được xử lý như một danh sách vòng, bộ điều phối lần lượt cấp phát cho từng tiến trình trong danh sách một khoảng thời gian sử dụng CPU gọi là quantum (lượng tử thời gian). Đây là một giải thuật điều phối không độc quyền: khi một tiến trình sử dụng CPU đến hết thời gian quantum dành cho nó, hệ điều hành thu hồi CPU và cấp cho tiến trình kế tiếp trong danh sách. Nếu tiến trình bị khóa hay kết thúc trước khi sử dụng hết thời gian quantum, hệ điều hành cũng lập tức cấp phát CPU cho tiến trình khác. Khi tiến trình tiêu thụ hết thời gian CPU dành cho nó mà chưa hoàn tất, tiến trình được đưa trở lại vào cuối danh sách sẵn sàng để đợi được cấp CPU trong lượt kế tiếp.

Ví dụ:



***Hình 3.2.3.2-1**. Lập lịch Round Robin*

Tiến trình	Thời điểm vào RL	Thời gian xử lý
P1	0	24
P2	1	3
P3	2	3

Nếu sử dụng quantum là 4 milisecondes, thứ tự cấp phát CPU sẽ là :

P1	P2	P3	P1	P1	P1	P1	P1
0	4	7	10	14	18	22	26 30

Thời gian chờ đợi trung bình sẽ là (0+6+3+5)/3 = 4.66 milisecondes. Nếu có n tiến trình trong danh sách sẵn sàng và sử dụng quantum q, thì mỗi tiến trình sẽ được cấp phát CPU 1/n trong từng khoảng thời gian q. Mỗi tiến trình sẽ không phải đợi quá (n-1)q đơn vị thời gian trước khi nhận được CPU cho lượt kế tiếp.

3.2.3.3 Lập lịch với độ ưu tiên

- **Nguyên tắc**: Mỗi tiến trình được gán cho một độ ưu tiên tương ứng, tiến trình có độ ưu tiên cao nhất sẽ được chọn để cấp phát CPU đầu tiên. Độ ưu tiên có thể được định nghĩa nội tại hay nhờ vào các yếu tố bên ngoài. Độ ưu tiên nội tại sử dụng các đại lượng có thể đo lường để tính toán độ ưu tiên của tiến trình, ví dụ các giới hạn thời gian, nhu cầu bộ nhớ… Độ ưu tiên cũng có thể được gán từ bên ngoài dựa vào các tiêu chuẩn do hệ điều hành như tầm quan trọng của tiến trình, loại người sử dụng sỡ hữu tiến trình… Giải thuật điều phối với độ ưu tiên có thể theo nguyên tắc độc quyền hay không độc quyền. Khi một tiến trình được đưa vào danh sách các tiến trình sẵn sàng, độ ưu tiên của nó được so sánh với độ ưu tiên của tiến trình hiện hành đang xử lý. Giải thuật điều phối với độ ưu tiên và không độc quyền sẽ thu hồi CPU từ tiến trình hiện hành để cấp phát cho tiến trình mới nếu độ ưu tiên của tiến trình này cao hơn tiến trình hiện hành. Một giải thuật độc quyền sẽ chỉ đơn giản chèn tiến trình mới

vào danh sách sẵn sàng, và tiến trình hiện hành vẫn tiếp tục xử lý hết thời gian dành cho nó.

Ví dụ: (độ ưu tiên 1 > độ ưu tiên 2> độ ưu tiên 3)

Tiến trình	Thời điểm vào RL	Độ ưu tiên	Thời gian xử lý
P1	0	3	24
P2	1	1	3
P3	2	2	3

Sử dụng thuật giải độc quyền, thứ tự cấp phát CPU như sau:

P1	P2	P3
0	24	27 30

Sử dụng thuật giải không độc quyền, thứ tự cấp phát CPU như sau:

P1	P2	P3	P1
0	1	4	7 30

3.2.3.4 Chiến lược công việc ngắn nhất (Shortest-job-first SJF)

– **Nguyên tắc**: Đây là một trường hợp đặc biệt của giải thuật điều phối với độ ưu tiên. Trong giải thuật này, độ ưu tiên p được gán cho mỗi tiến trình là nghịch đảo của thời gian xử lý t mà tiến trình yêu cầu: p = 1/t. Khi CPU được tự do, nó sẽ được cấp phát cho tiến trình yêu cầu ít thời gian nhất để kết thúc- tiến trình ngắn nhất. Giải thuật này cũng có thể độc quyền hay không độc quyền. Sự chọn lựa xảy ra khi có một tiến trình mới được đưa vào danh sách sẵn sàng trong khi một tiến trình khác đang xử lý. Tiến trình mới có thể sở hữu một yêu cầu thời gian sử dụng CPU cho lần tiếp theo (CPU-burst) ngắn hơn thời gian còn lại mà tiến trình hiện hành cần xử lý. Giải thuật SJF không độc quyền sẽ dừng hoạt động của tiến trình hiện hành, trong khi giải thuật độc quyền sẽ cho phép tiến trình hiện hành tiếp tục xử lý.

Ví dụ:

Tiến trình	Thời điểm vào RL	Thời gian xử lý
P1	0	6
P2	1	8
P3	2	4
P4	3	2

Sử dụng thuật giải SJF độc quyền, thứ tự cấp phát CPU như sau:

P1	P4	P3	P2
0	6	8	12 30

Sử dụng thuật giải SJF không độc quyền, thứ tự cấp phát CPU như sau:

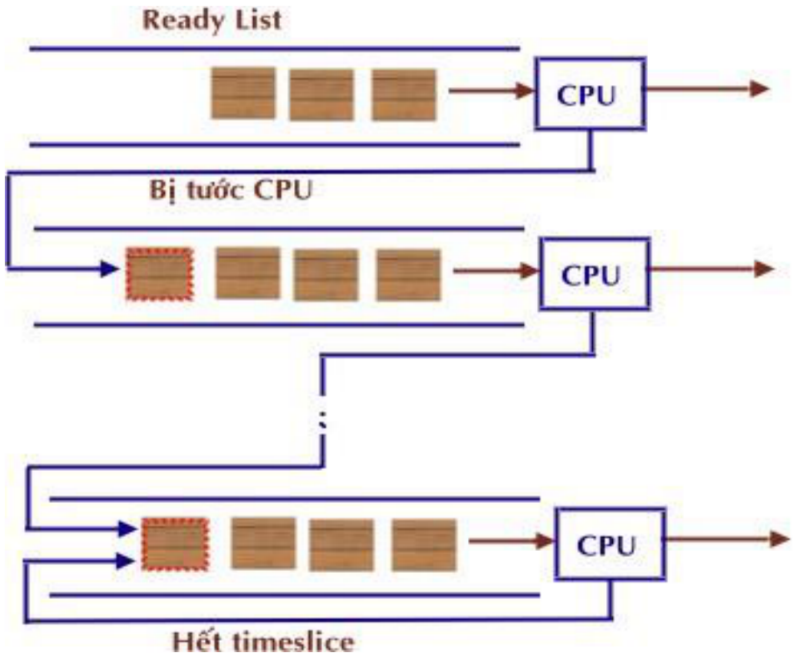
P1	P4	P1	P3	P2
0	3	5	8	12 20

3.2.3.5 Chiến lược điều phối với nhiều mức độ ưu tiên

- **Nguyên tắc**: Ý tưởng chính của giải thuật là phân lớp các tiến trình tùy theo độưu tiên của chúng để có cách thức điều phối thích hợp cho từng nhóm. Danh sách sẵn sàng được phân tách thành các danh sách riêng biệt theo cấp độ ưu tiên, mỗi danh sách bao gồm các tiến trình có cùng độ ưu tiên và được áp dụng một giải thuật điều phối thích hợp để điều phối. Ngoài ra, còn có một giải thuật điều phối giữa các nhóm, thường giải thuật này là giải thuật không độc quyền và sử dụng độ ưu tiên cố định.Một tiến trình thuộc về danh sách ở cấp ưu tiên i sẽ chỉ được cấp phát CPU khi các danh sách ở cấp ưu tiên lớn hơn i đã trống.



Hình 3.2.3.5-1. *Điều phối nhiều cấp ưu tiên*



Hình 3.2.3.5-2*. Điều phối Multilevel Feedback*

3.2.3.6 Chiến lược lập lịch Xổ số (Lottery)

- **Nguyên tắc**: Ý tưởng chính của giải thuật là phát hành một số vé số và phân phối cho các tiến trình trong hệ thống. Khi đến thời điểm ra quyết định điều phối, sẽ tiến hành chọn 1 vé "trúng giải", tiến trình nào sỡ hữu vé này sẽ được nhận CPU(chọn ngẫu nhiên)

Câu hỏi ôn tập

1. Nêu các mục tiêu của lập lịch tiến trình.
2. Phân biệt điều phối độc quyền và điều phối không độc quyền.
3. Trình bày các danh sách sử dụng trong quá trình lập lịch và các cấp độ lập lịch.
4. Cho 3 tiến trình P1, P2, P3 có thời điểm vào và thời gian xử lý tùy chọn, hãy tính thời gian chờ trung bình theo FIFO, Round Robin (quantum = 2) và SJF.
5. Trình bày chiến lược điều phối với nhiều mức độ ưu tiên và chiến lược xổ số (Lottery).

Bài 7: Lập trình tiến trình — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: chương trình cha kết thúc khi công việc con chưa xong hoặc không thu kết quả. Phải xác định cách tạo, chờ, trao đổi trạng thái thoát và giải phóng tài nguyên của tiến trình. API cụ thể phụ thuộc hệ điều hành và ngôn ngữ.

Tự luyện: mô tả cha tạo hai tác vụ lab, chờ cả hai và ghi lỗi nếu một tác vụ thất bại. **Hướng giải:** tách tạo tiến trình khỏi việc chờ; không coi in “đã tạo” là công việc đã hoàn thành. Không dùng ví dụ fork trên Windows mà không giải thích nền tảng hoặc lớp trừu tượng thay thế.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 4 · BÀI 1

4.1. Liên lạc tiến trình

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Hiểu vì sao các tiến trình cần liên lạc với nhau.
- Nhận biết các vấn đề nảy sinh khi các tiến trình liên lạc.

4.1.1 Nhu cầu liên lạc tiến trình

Trong môi trường đa chương, một tiến trình không đơn độc trong hệ thống, mà có thể ảnh hưởng đến các tiến trình khác, hoặc bị các tiến trình khác tác động. Nói cách khác, các tiến trình là những thực thể độc lập, nhưng chúng vẫn có nhu cầu liên lạc với nhau để :

- Chia sẻ thông tin: nhiều tiến trình có thể cùng quan tâm đến những dữ liệu nào đó, do vậy hệ điều hành cần cung cấp một môi trường cho phép sự truy cập đồng thời đến các dữ liệu chung.
- Hợp tác hoàn thành tác vụ: đôi khi để đạt được một sự xử lý nhanh chóng, người ta phân chia một tác vụ thành các công việc nhỏ có thể tiến hành song song. Thường thì các công việc nhỏ này cần hợp tác với nhau để cùng hoàn thành tác vụ ban đầu, ví dụ dữ liệu kết xuất của tiến trình này lại là dữ liệu nhập cho tiến trình khác … Trong các trường hợp đó, hệ điều hành cần cung cấp cơ chế để các tiến trình có thể trao đổi thông tin với nhau.

4.1.2 Các vấn đề nảy sinh trong việc liên lạc tiến trình

Do mỗi tiến trình sỡ hữu một không gian địa chỉ riêng biệt, nên các tiến trình không thể liên lạc trực tiếp dễ dàng mà phải nhờ vào các cơ chế do hệ điều hành cung cấp. Khi cung cấp cơ chế liên lạc cho các tiến trình, hệ điều hành thường phải tìm giải pháp cho các vấn đề chính yếu sau:

- Liên kết tường minh hay tiềm ẩn (explicit naming/implicit naming): tiến trình có cần phải biết tiến trình nào đang trao đổi hay chia sẻ thông tin với nó? Mối liên kết được gọi là tường minh khi được thiết lập rõ ràng, trực tiếp giữa các tiến trình, và là tiềm ẩn khi các tiến trình liên lạc với nhau thông qua một qui ước ngầm nào đó.
- Liên lạc theo chế độ đồng bộ hay không đồng bộ (blocking/non-blocking): khi một tiến trình trao đổi thông tin với một tiến trình khác, các tiến trình có cần phải đợi cho thao tác liên lạc hoàn tất rồi mới tiếp tục các xử lý khác? Các tiến trình liên lạc theo cơ chế đồng bộ sẽ chờ nhau hoàn tất việc liên lạc, còn các tiến trình liên lạc theo cơ

chế nonblocking thì không.

– Liên lạc giữa các tiến trình trong hệ thống tập trung và hệ thống phân tán: cơ chế liên lạc giữa các tiến trình trong cùng một máy tính có sự khác biệt với việc liên lạc giữa các tiến trình giữa những máy tính khác nhau? Hầu hết các hệ điều hành đưa ra nhiều cơ chế liên lạc khác nhau, mỗi cơ chế có những đặc tính riêng, và thích hợp trong một hoàn cảnh chuyên biệt.

Câu hỏi ôn tập

1. Nêu các lý do khiến các tiến trình cần liên lạc với nhau.
2. Những vấn đề nào nảy sinh trong việc liên lạc tiến trình?

Bài 8: Liên lạc tiến trình — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: hai chương trình cần trao đổi kết quả. Chọn cơ chế dựa vào lượng dữ liệu, yêu cầu độ trễ, phạm vi máy và đồng bộ: pipe, hàng đợi thông điệp, socket hoặc vùng nhớ chia sẻ có thể phù hợp các bài khác nhau.

Tự luyện: thiết kế thông điệp gồm loại, id yêu cầu và dữ liệu cho hai tiến trình lab. **Hướng giải:** quy định thông điệp lỗi, bên đóng kết nối và tình huống bên kia dừng. Vùng nhớ chia sẻ giảm việc sao chép trong một số thiết kế nhưng không tự giải quyết đồng bộ hoặc tính hợp lệ dữ liệu.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 4 · BÀI 2

4.2. Các cơ chế thông tin liên lạc

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Nắm được các cơ chế liên lạc: tín hiệu, pipe, vùng nhớ chia sẻ, trao đổi thông điệp, socket.
- So sánh ưu nhược điểm của từng cơ chế.
- Biết khi nào nên dùng cơ chế nào.

4.2.1 Tín hiệu (Signal)

Giới thiệu: Tín hiệu là một cơ chế phần mềm tương tự như các ngắt cứng tác động đến các tiến trình. Một tín hiệu được sử dụng để thông báo cho tiến trình về một sự kiện nào đó xảy ra. Có nhiều tín hiệu được định nghĩa, mỗi một tín hiệu có một ý nghĩa tương ứng với một sự kiện đặc trưng.

Ví dụ : Một số tín hiệu của UNIX

Tín hiệu	Mô tả
SIGINT	Người dùng nhấn phím DEL để ngắt xử lý tiến trình
SIGQUIT	Yêu cầu thoát xử lý
SIGILL	Tiến trình xử lý một chỉ thị bất hợp lệ
SIGKILL	Yêu cầu kết thúc một tiến trình
SIGFPT	Lỗi floating – point xảy ra (chia cho 0)
SIGPIPE	Tiến trình ghi dữ liệu vào pipe mà không có reader
SIGSEGV	Tiến trình truy xuất đến một địa chỉ bất hợp lệ
SIGCLD	Tiến trình con kết thúc

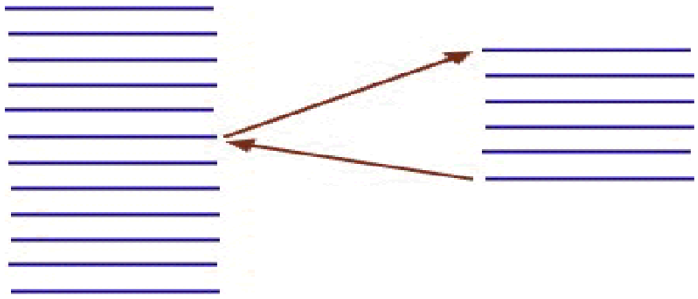
SIGUSR1	Tín hiệu 1 do người dùng định nghĩa
SIGUSR2	Tín hiệu 2 do người dùng định nghĩa

Mỗi tiến trình sỡ hữu một bảng biểu diễn các tín hiệu khác nhau. Với mỗi tín hiệu sẽ có tương ứng một trình xử lý tín hiệu (signal handler) qui định các xử lý của tiến trình khi nhận được tín hiệu tương ứng. Các tín hiệu được gởi đi bởi:

- Phần cứng (ví dụ lỗi do các phép tính số học)
- Hạt nhân hệ điều hành gởi đến một tiến trình (ví dụ lưu ý tiến trình khi có một thiết bị nhập/xuất tự do).
- Một tiến trình gởi đến một tiến trình khác (ví dụ tiến trình cha yêu cầu một tiến trình con kết thúc)
- Người dùng (ví dụ nhấn phím Ctl-C để ngắt xử lý của tiến trình)

Khi một tiến trình nhận một tín hiệu, nó có thể xử sự theo một trong các cách sau:

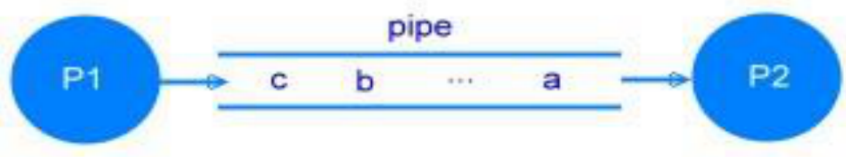
- Bỏ qua tín hiệu
- Xử lý tín hiệu theo kiểu mặc định
- Tiếp nhận tín hiệu và xử lý theo cách đặc biệt của tiến trình.



Hình 4.2.1-1. *Liên lạc bằng tín hiệu*

4.2.2 Pipe

Giới thiệu: Một pipe là một kênh liên lạc trực tiếp giữa hai tiến trình : dữ liệu xuất của tiến trình này được chuyển đến làm dữ liệu nhập cho tiến trình kia dưới dạng một dòng các byte. Khi một pipe được thiết lập giữa hai tiến trình, một trong chúng sẽ ghi dữ liệu vào pipe và tiến trình kia sẽ đọc dữ liệu từ pipe. Thứ tự dữ liệu truyền qua pipe được bảo toàn theo nguyên tắc FIFO. Một pipe có kích thước giới hạn (thường là 4096 ký tự)



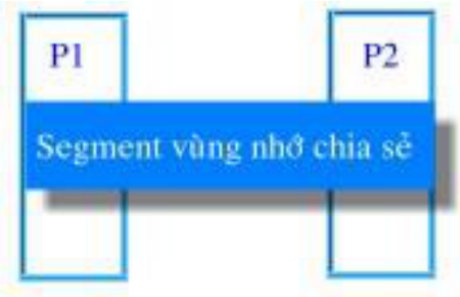
Hình 4.2.2-1*. Liên lạc qua pipe*

Một tiến trình chỉ có thể sử dụng một pipe do nó tạo ra hay kế thừa từ tiến trình cha. Hệ điều hành cung cấp các lời gọi hệ thống read/write cho các tiến trình thực hiện thao tác đọc/ghi dữ liệu trong pipe. Hệ điều hành cũng chịu trách nhiệm đồng bộ hóa việc truy xuất pipe trong các tình huống:

- Tiến trình đọc pipe sẽ bị khóa nếu pipe trống, nó sẽ phải đợi đến khi pipe có dữ liệu để truy xuất.
- Tiến trình ghi pipe sẽ bị khóa nếu pipe đầy, nó sẽ phải đợi đến khi pipe có chỗ trống để chứa dữ liệu. Đọc hoặc ghi, nhưng không thể thực hiện cả hai. Một số hệ điều hành cho phép thiết lập hai pipe giữa một cặp tiến trình để tạo liên lạc hai chiều. Trong những hệ thống đó, có nguy cơ xảy ra tình trạng tắc nghẽn (deadlock): một pipe bị giới hạn về kích thước, do vậy nếu cả hai pipe nối kết hai tiến trình đều đầy(hoặc đều trống) và cả hai tiến trình đều muốn ghi (hay đọc) dữ liệu vào pipe (mỗi tiến trình ghi dữ liệu vào một pipe), chúng sẽ cùng bị khóa và chờ lẫn nhau mãi mãi. Cơ chế này cho phép truyền dữ liệu với cách thức không cấu trúc. Ngoài ra, một giới hạn của hình thức liên lạc này là chỉ cho phép kết nối hai tiến trình có quan hệ cha-con, và trên cùng một máy tính.

4.2.3 Vùng nhớ chia sẻ

Giới thiệu: Cách tiếp cận của cơ chế này là cho nhiều tiến trình cùng truy xuất đến một vùng nhớ chung gọi là vùng nhớ chia sẻ (shared memory). Không có bất kỳ hành vi truyền dữ liệu nào cần phải thực hiện ở đây, dữ liệu chỉ đơn giản được đặt vào một vùng nhớ mà nhiều tiến trình có thể cùng truy cập được. Với phương thức này, các tiến trình chia sẻ một vùng nhớ vật lý thông qua trung gian không gian địa chỉ của chúng. Một vùng nhớ chia sẻ tồn tại độc lập với các tiến trình, và khi một tiến trình muốn truy xuất đến vùng nhớ này, tiến trình phải kết gắn vùng nhớ chung đó vào không gian địa chỉ riêng của từng tiến trình, và thao tác trên đó như một vùng nhớ riêng của mình.



***Hình 4.2.3-1**. Liên lạc qua vùng nhớ chia sẻ*

4.2.4 Trao đổi thông điệp (Message)

Giới thiệu: Hệ điều hành còn cung cấp một cơ chế liên lạc giữa các tiến trình không thông qua việc chia sẻ một tài nguyên chung , mà thông qua việc gởi thông điệp. Để hỗ trợ cơ chế liên lạc bằng thông điệp, hệ điều hành cung cấp các hàm IPC chuẩn (Interprocess communication), cơ bản là hai hàm:

- Send (message): gởi một thông điệp
- Receive (message): nhận một thông điệp

Nếu hai tiến trình P và Q muốn liên lạc với nhau, cần phải thiết lập một mối liên kết giữa hai tiến trình, sau đó P, Q sử dụng các hàm IPC thích hợp để trao đổi thông điệp, cuối cùng khi sự liên lạc chấm dứt mối liên kết giữa hai tiến trình sẽ bị hủy. Có nhiều cách thức để thực hiện sự liên kết giữa hai tiến trình và cài đặt các theo tác send /receive tương ứng: liên lạc trực tiếp hay gián tiếp, liên lạc đồng bộ hoặc không đồng bộ, kích thước thông điệp là cố định hay không … Nếu các tiến trình liên lạc theo kiểu liên kết tường minh, các hàm Send và Receive sẽ được cài đặt với tham số:

- Send(destination, message): gởi một thông điệp đến destination
- Receive(source,message): nhận một thông điệp từ source

4.2.5 Sockets

Giới thiệu: Một socket là một thiết bị truyền thông hai chiều tương tự như tập tin, chúng ta có thể đọc hay ghi lên nó, tuy nhiên mỗi socket là một thành phần trong một mối nối nào đó giữa các máy trên mạng máy tính và các thao tác đọc/ghi chính là sự trao đổi dữ liệu giữa các ứng dụng trên nhiều máy khác nhau. Sử dụng socket có thể mô phỏng hai phương thức liên lạc trong thực tế: liên lạc thư tín (socket đóng vai trò bưu cục) và liên lạc điện thoại (socket đóng vai trò tổng đài). Các thuộc tính của socket:

- Domaine: định nghĩa dạng thức địa chỉ và các nghi thức sử dụng. Có nhiều domaines, ví dụ UNIX, INTERNET, XEROX_NS, ...
- Type: định nghĩa các đặc điểm liên lạc:

  a) Sự tin cậy

  b) Sự bảo toàn thứ tự dữ liệu

  c) Lặp lại dữ liệu

  d) Chế độ nối kết

  e) Bảo toàn giới hạn thông điệp

  f) Khả năng gởi thông điệp khẩn

Để thực hiện liên lạc bằng socket, cần tiến hành các thao tác:

- Tạo lập hay mở một socket
- Gắn kết một socket với một địa chỉ
- Liên lạc: có hai kiểu liên lạc tùy thuộc vào chế độ nối kết:

a) Liên lạc trong chế độ không liên kết: liên lạc theo hình thức hộp thư:

- Hai tiến trình liên lạc với nhau không kết nối trực tiếp
- Mỗi thông điệp phải kèm theo địa chỉ người nhận.

Hình thức liên lạc này có đặc điểm được:

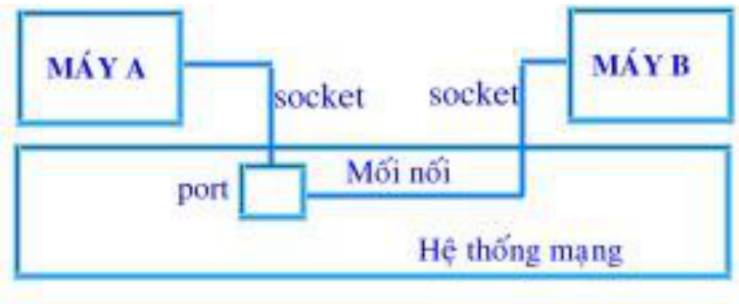
- Người gởi không chắc chắn thông điệp của học được gởi đến người nhận,
- Một thông điệp có thể được gởi nhiều lần,
- Hai thông điệp đượ gởi theo một thứ tự nào đó có thể đến tay người nhận theo một thứ tự khác.

Một tiến trình sau khi đã mở một socket có thể sử dụng nó để liên lạc với nhiều tiến trình khác nhau nhờ sử hai primitive send và receive.

b) Liên lạc trong chế độ nối kết: Một liên kết được thành lập giữa hai tiến trình. Trước khi mối liên kết này được thiết lập, một trong hai tiến trình phải đợi có một tiến trình khác yêu cầu kết nối.Có thể sử dụng socket để liên lạc theo mô hình client-serveur. Trong mô hình này, server sử dụng lời gọi hệ thống listen và accept để nối kết với client, sau đó, client và server có thể trao đổi thông tin bằng cách sử dụng các primitive send và receive.

Hủy một socket

Ví dụ: Trong nghi thức truyền thông TCP, mỗi mối nối giữa hai máy tính được xác định bởi một port, khái niệm port ở đây không phải là một cổng giao tiếp trên thiết bị vật lý mà chỉ là một khái niệm logic trong cách nhìn của người lập trình, mỗi port được tương ứng với một số nguyên dương.



***Hình 4.2.5-1**. Các socket và port trong mối nối TCP.*

Hình 4.2.5-1 minh họa một cách giao tiếp giữa hai máy tính trong nghi thức truyền thông TCP. Máy A tạo ra một socket và kết buộc (bind) socket nầy với một port X (tức là một số nguyên dương có ý nghĩa

cục bộ trong máy A), trong khi đó máy B tạo một socket khác và móc vào (connect) port X trong máy A.

Câu hỏi ôn tập

1. Tín hiệu (signal) là gì? Tiến trình có thể xử lý một tín hiệu theo những cách nào?
2. Trình bày cơ chế pipe và các hạn chế của nó.
3. So sánh vùng nhớ chia sẻ và trao đổi thông điệp.
4. Socket là gì? Vì sao socket phù hợp cho liên lạc giữa các tiến trình trên các máy khác nhau?

Bài 9: Các cơ chế thông tin liên lạc — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: gửi một tín hiệu không đồng nghĩa đã chuyển toàn bộ một bản ghi dữ liệu. Cơ chế thông báo sự kiện khác với giao thức truyền dữ liệu có cấu trúc; cần hiểu giới hạn và thứ tự xử lý của API cụ thể.

Tự luyện: mô tả gửi thông báo “có dữ liệu mới” và cách nhận dữ liệu thật từ hàng đợi. **Hướng giải:** bên nhận phải xử lý trường hợp trễ, đóng và lỗi; không làm thao tác phức tạp trong signal handler mà không kiểm tra quy tắc nền tảng. Xác định quyền và định danh tiến trình để tránh gửi nhầm.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 4 · BÀI 3

4.3. Nhu cầu đồng bộ hóa (synchronisation)

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Hiểu hai yêu cầu đồng bộ: độc quyền truy xuất và phối hợp.
- Nắm được khái niệm tranh đoạt điều khiển (race condition) và miền găng (critical section).
- Biết các điều kiện một giải pháp đồng bộ tốt phải thỏa mãn.

Trong một hệ thống cho phép các tiến trình liên lạc với nhau, bao giờ hệ điều hành cũng cần cung cấp kèm theo những cơ chế đồng bộ hóa để bảo đảm hoạt động của các tiến trình đồng hành không tác động sai lệch đến nhau vì các lý do sau đây:

4.3.1 Yêu cầu độc quyền truy xuất (Mutual exclusion)

Các tài nguyên trong hệ thống được phân thành hai loại: tài nguyên có thể chia sẻ cho phép nhiều tiến trình đồng thời truy xuất, và tài nguyên không thể chia sẻ chỉ chấp nhận một (hay một số lượng hạn chế) tiến trình sử dụng tại một thời điểm. Tính không thể chia sẻ của tài nguyên thường có nguồn gốc từ một trong hai nguyên nhân sau đây:

– Đặc tính cấu tạo phần cứng của tài nguyên không cho phép chia sẻ.

– Nếu nhiều tiến trình sử dụng tài nguyên đồng thời, có nguy cơ xảy ra các kết quả không dự đoán được do hoạt động của các tiến trình trên tài nguyên ảnh hưởng lẫn nhau.

Để giải quyết vấn đề, cần bảo đảm tiến trình độc quyền truy xuất tài nguyên, nghĩa là hệ thống phải kiểm soát sao cho tại một thời điểm, chỉ có một tiến trình được quyền truy xuất một tài nguyên không thể chia sẻ.

4.3.2 Yêu cầu phối hợp (Synchronization)

Nhìn chung, mối tương quan về tốc độ thực hiện của hai tiến trình trong hệ thống là không thể biết trước, vì điều này phụ thuộc vào nhiều yếu tố động như tần suất xảy ra các ngắt của từng tiến trình, thời gian tiến trình được cấp phát bộ xử lý… Có thể nói rằng các tiến trình hoạt động không đồng bộ với nhau. Nhưng có những tình huống các tiến trình cần hợp tác trong việc hoàn thành tác vụ, khi đó cần phải đồng bộ hóa hoạt động của các tiến trình, ví dụ một tiến trình chỉ có thể xử lý nếu một tiến trình khác đã kết thúc một công việc nào đó …

4.3.3 Bài toán đồng bộ hoá

4.3.3.1 Vấn đề tranh đoạt điều khiển (race condition)

Giả sử có hai tiến trình P1 và P2 thực hiện công việc của các kế toán, và cùng chia sẻ một vùng nhớ chung lưu trữ biến taikhoan phản ánh thông tin về tài khoản. Mỗi tiến trình muốn rút một khoản tiền tienrut từ tài khoản:

```
if (taikhoan - tienrut >=0)
taikhoan = taikhoan - tienrut;
else
error(“khong the rut tien !”);
```

Giả sử trong tài khoản hiện còn 800, P1 muốn rút 500 và P2 muốn rút 400. Nếu xảy ra tình huống như sau:

- Sau khi đã kiểm tra điều kiện (taikhoan - tienrut >=0) và nhận kết quả là 300, P1 hết thời gian xử lý mà hệ thống cho phép, hệ điều hành cấp phát CPU cho P2.
- P2 kiểm tra cùng điều kiện trên, nhận được kết quả là 400 (do P1 vẫn chưa rút tiền) và rút 400. Giá trị của taikhoan được cập nhật lại là 400.
- Khi P1 được tái kích hoạt và tiếp tục xử lý, nó sẽ không kiểm tra lại điều kiện (taikhoan - tienrut >=0)-vì đã kiểm tra trong lượt xử lý trước- mà thực hiện rút tiền. Giá trị của taikhoan sẽ lại được cập nhật thành -100. Tình huống lỗi xảy ra. Các tình huống tương tự như thế - có thể xảy ra khi có nhiều hơn hai tiến trình đọc và ghi dữ liệu trên cùng một vùng nhớ chung, và kết quả phụ thuộc vào sự điều phối tiến trình của hệ thống- được gọi là các tình huống tranh đoạt điều khiển (race condition) .

4.3.3.2 Miền găng (critical section)

Để ngăn chặn các tình huống lỗi có thể nảy sinh khi các tiến trình truy xuất đồng thời một tài nguyên không thể chia sẻ, cần phải áp đặt một sự truy xuất độc quyền trên tài nguyên đó: khi một tiến trình đang sử dụng tài nguyên, thì những tiến trình khác không được truy xuất đến tài nguyên.

Đoạn chương trình trong đó có khả năng xảy ra các mâu thuẫn truy xuất trên tài nguyên chung được gọi là miền găng (critical section). Trong ví dụ trên, đoạn mã:

```
if (taikhoan - tienrut >=0)
taikhoan = taikhoan - tienrut;
```

của mỗi tiến trình tạo thành một miền găng.

Có thể giải quyết vấn đề mâu thuẫn truy xuất nếu có thể bảo đảm tại một thời điểm chỉ có duy nhất một tiến trình được xử lý lệnh trong miền găng. Một phương pháp giải

quyết tốt bài toán miền găng cần thõa mãn 4 điều kiện sau:

- Không có hai tiến trình cùng ở trong miền găng cùng lúc.
- Không có giả thiết nào đặt ra cho sự liên hệ về tốc độ của các tiến trình, cũng như về số lượng bộ xử lý trong hệ thống.
- Một tiến trình tạm dừng bên ngoài miền găng không được ngăn cản các tiến trình khác vào miền găng.
- Không có tiến trình nào phải chờ vô hạn để được vào miền găng.

Câu hỏi ôn tập

1. Thế nào là độc quyền truy xuất (mutual exclusion)? Cho ví dụ.
2. Race condition là gì? Nêu một ví dụ cụ thể.
3. Miền găng là gì? Một giải pháp giải quyết bài toán miền găng phải thỏa mãn những điều kiện nào?

Bài 10: Nhu cầu đồng bộ hóa — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: hai thread cùng tăng số lượt đăng ký. Phép đọc–cộng–ghi gồm nhiều bước; nếu cùng đọc 10 rồi đều ghi 11 thì mất một cập nhật. Đây là mô hình minh họa race condition, không phải bảo đảm mọi runtime đều xen kẽ đúng như vậy.

Tự luyện: viết các bước xen kẽ gây kết quả sai và đánh dấu vùng tới hạn. **Hướng giải:** tìm dữ liệu dùng chung và bất biến cần giữ, rồi chọn đồng bộ phù hợp. Deadlock khác race: ví dụ T1 giữ A chờ B, T2 giữ B chờ A nên cả hai không tiến được.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/

CHƯƠNG 4 · BÀI 4

4.4. Các giải pháp đồng bộ hóa

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Phân biệt hai nhóm giải pháp đồng bộ: busy waiting và sleep & wakeup.
- Hiểu và phân tích được biến khóa, kiểm tra luân phiên, Peterson, cấm ngắt, TSL.
- Sử dụng được semaphore, monitor và trao đổi thông điệp để đồng bộ tiến trình.

4.4.1 Giải pháp "busy waiting"

4.4.1.1 Sử dụng các biến cờ hiệu(simaphore)

Tiếp cận: các tiến trình chia sẻ một biến chung đóng vai trò "chốt cửa" (lock), biến này được khởi động là 0. Một tiến trình muốn vào miền găng trước tiên phải kiểm tra giá trị của biến lock. Nếu lock = 0, tiến trình đặt lại giá trị cho lock = 1 và đi vào miền găng. Nếu lock đang nhận giá trị 1, tiến trình phải chờ bên ngoài miền găng cho đến khi lock có giá trị 0. Như vậy giá trị 0 của lock mang ý nghĩa là không có tiến trình nào đang ở trong miền găng, và lock = 1 khi có một tiến trình đang ở trong miền găng.

```
while (TRUE) {
while (lock == 1); // wait
lock = 1;
critical-section ();
lock = 0;
Noncritical-section (); }
```

4.4.1.2 Sử dụng việc kiểm tra luân phiên

Tiếp cận: Đây là một giải pháp đề nghị cho hai tiến trình. Hai tiến trình này sử dụng chung biến turn (phản ánh phiên tiến trình nào được vào miền găng), được khởi động với giá trị 0. Nếu turn = 0, tiến trình A được vào miền găng. Nếu turn = 1, tiến trình A đi vào một vòng lặp chờ đến khi turn nhận giá trị 0. Khi tiến trình A rời khỏi miền găng, nó đặt giá trị turn về 1 để cho phép tiến trình B đi vào miền găng.

```
while (TRUE) {
while (turn != 0); // wait
critical-section ();
turn = 1;
Noncritical-section (); }
```

(a) Cấu trúc tiến trình A

```
while (TRUE) {
while (turn != 1); // wait
critical-section ();
turn = 0;
Noncritical-section (); }
```

4.4.1.3 Giải pháp của Peterson

Tiếp cận: Petson đưa ra một giải pháp kết hợp ý tưởng của cả hai giải pháp kể trên. Các tiến trình chia sẻ hai biến chung:

```
int turn; // đến phiên ai
int interesse[2]; // khởi động là FALSE
```

Nếu interesse[i] = TRUE có nghĩa là tiến trình Pi muốn vào miền găng. Khởi đầu, interesse[0] = interesse[1] = FALSE và giá trị của est được khởi động là 0 hay 1. Để có thể vào được miền găng, trước tiên tiến trình Pi đặt giá trị interesse[i]=TRUE (xác định rằng tiến trình muốn vào miền găng), sau đó đặt turn=j (đề nghị thử tiến trình khác vào miền găng). Nếu tiến trình Pj không quan tâm đến việc vào miền găng (interesse[j]=FALSE), thì Pi có thể vào miền găng, nếu không, Pi phải chờ đến khi interesse[j]=FALSE. Khi tiến trình Pi rời khỏi miền găng, nó đặt lại giá trị cho interesse[i]= FALSE.

```
while (TRUE) {
int j = 1-i; // j là tiến trình còn lại
interesse[i]= TRUE;
turn = j;
while (turn == j && interesse[j]==TRUE);
critical-section ();
interesse[i] = FALSE;
Noncritical-section (); }
```

4.4.1.4 Cấm ngắt

Là giải pháp phần cứng.

Tiếp cận: cho phép tiến trình cấm tất cả các ngắt trước khi vào miền găng, và phục hồi ngắt khi ra khỏi miền găng. Khi đó, ngắt đồng hồ cũng không xảy ra, do vậy hệ thống không thể tạm dừng hoạt động của tiến trình đang xử lý để cấp phát CPU cho tiến trình khác, nhờ đó tiến trình hiện hành yên tâm thao tác trên miền găng mà không sợ bị tiến trình nào khác tranh chấp.

4.4.1.5 Chỉ thị TSL (Test-and-Set)

Là giải pháp phần cứng.

Tiếp cận: đây là một giải pháp đòi hỏi sự trợ giúp của cơ chế phần cứng. Nhiều máy tính cung cấp một chỉ thị đặc biệt cho phép kiểm tra và cập nhật nội dung một vùng nhớ trong một thao tác không thể phân chia, gọi là chỉ thị Test-and- Set Lock (TSL) và được định nghĩa như sau:

Test-and-Setlock (boolean target)

```
{ Test-and-Setlock = target;
target = TRUE; }
```

Nếu có hai chỉ thị TSL xử lý đồng thời (trên hai bộ xử lý khác nhau), chúng sẽ được xử lý tuần tự. Có thể cài đặt giải pháp truy xuất độc quyền với TSL bằng cách sử dụng thêm một biến lock, được khởi gán là FALSE. Tiến trình phải kiểm tra giá trị của biến lock trước khi vào miền găng, nếu lock = FALSE, tiến trình có thể vào miền găng.

```
while (TRUE) {
while (Test-and-Setlock(lock));
critical-section ();
lock = FALSE;
Noncritical-section (); }
```

Tất cả các giải pháp trên đây đều phải thực hiện một vòng lặp để kiểm tra liệu nó có được phép vào miền găng, nếu điều kiện chưa cho phép, tiến trình phải chờ tiếp tục trong vòng lặp kiểm tra này. Các giải pháp buộc tiến trình phải liên tục kiểm tra điều kiện để phát hiện thời điểm thích hợp được vào miền găng như thế được gọi các giải pháp “busy waiting”. Lưu ý rằng việc kiểm tra như thế tiêu thụ rất nhiều thời gian sử dụng CPU, do vậy tiến trình đang chờ vẫn chiếm dụng CPU. Xu hướng giải quyết vấn đề đồng bộ hoá là nên tránh các giải pháp “busy waiting”.

4.4.2 Các giải pháp “SLEEP and WAKEUP”

Để loại bỏ các bất tiện của giải pháp “busy waiting”, chúng ta có thể tiếp cận theo hướng cho một tiến trình chưa đủ điều kiện vào miền găng chuyển sang trạng thái blocked, từ bỏ quyền sử dụng CPU. Để thực hiện điều này, cần phải sử dụng các thủ tục do hệ điều hành cung cấp để thay đổi trạng thái tiến trình. Hai thủ tục cơ bản SLEEP và WAKEUP thường được sử dụng để phục vụ mục đích này. SLEEP là một lời gọi hệ thống có tác dụng tạm dừng hoạt động của tiến trình (blocked) gọi nó và chờ đến khi được một tiến trình khác “đánh thức”. Lời gọi hệ thống WAKEUP nhận một tham số duy nhất: tiến trình sẽ được tái kích hoạt (đặt về trạng thái ready).

Ý tưởng sử dụng SLEEP và WAKEUP như sau: khi một tiến trình chưa đủ điều kiện vào miền găng, nó gọi SLEEP để tự khóa đến khi có một tiến trình khác gọi WAKEUP để giải phóng cho nó. Một tiến trình gọi WAKEUP khi ra khỏi miền găng để đánh thức một tiến trình đang chờ, tạo cơ hội cho tiến trình này vào miền găng:

```
int busy; // 1 nếu miền găng đang bị chiếm, nếu không là 0
int blocked; // đếm số lượng tiến trình đang bị khóa
while (TRUE) {
if (busy){
blocked = blocked + 1;
sleep();
} else busy = 1;
critical-section ();
busy = 0;
if(blocked){
wakeup(process);
blocked = blocked - 1; }
Noncritical-section (); }
```

Khi sử dụng SLEEP và WAKEUP cần hết sức cẩn thận, nếu không muốn xảy ra tình trạng mâu thuẫn truy xuất trong một vài tình huống đặc biệt như sau: giả sử tiến trình A vào miền găng, và trước khi nó rời khỏi miền găng thì tiến trình B được kích hoạt. Tiến trình B thử vào miền găng nhưng nó nhận thấy A đang ở trong đó, do vậy B tăng giá trị biến blocked và chuẩn bị gọi SLEEP để tự khoá. Tuy nhiên trước khi B có thể thực hiện SLEEP, tiến trình A lại được tái kích hoạt và ra khỏi miền găng. Khi ra khỏi miền găng A nhận thấy có một tiến trình đang chờ (blocked=1) nên gọi WAKEUP và giảm giá trị của blocked. Khi đó tín hiệu WAKEUP sẽ lạc mất do tiến trình B chưa thật sự “ngủ” để nhận tín hiệu đánh thức. Khi tiến trình B được tiếp tục xử lý, nó mới gọi SLEEP và tự khó vĩnh viễn. Vấn đề ghi nhận được là tình trạng lỗi này xảy ra do việc kiểm tra tư cách vào miền găng và việc gọi SLEEP hay WAKEUP là những hành động tách biệ, có thể bị ngắt nửa chừng trong quá trình xử lý, do đó có khi tín hiệu WAKEUP gởi đến một tiến trình chưa bị khóa sẽ lạc mất.

Để tránh những tình huống tương tự, hệ điều hành cung cấp những cơ chế đồng bộ hóa dựa trên ý tưởng của chiến lược “SLEEP and WAKEUP” nhưng được xây dựng bao hàm cả phương tiện kiểm tra điều kiện vào miền găng giúp sử dụng an toàn.

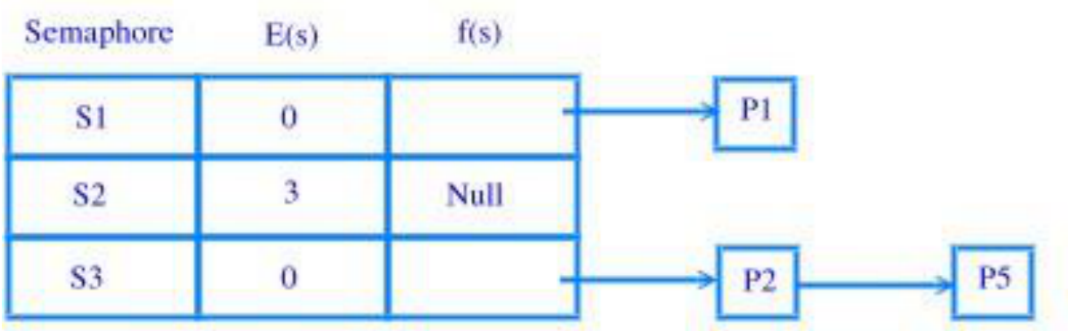
4.4.2.1 Semaphore

Tiếp cận: Được Dijkstra đề xuất vào 1965, một semaphore s là một biến có các thuộc tính sau:

- Một giá trị nguyên dương e(s)
- Một hàng đợi f(s) lưu danh sách các tiến trình đang bị khóa (chờ) trên semaphores
- Chỉ có hai thao tác được định nghĩa trên semaphore

Down(s): giảm giá trị của semaphore s đi 1 đơn vị nếu semaphore có trị e(s) > 0, và tiếp tục xử lý. Ngược lại, nếu e(s) 0, tiến trình phải chờ đến khi e(s) >0.

Up(s): tăng giá trị của semaphore s lên 1 đơn vị. Nếu có một hoặc nhiều tiến trình đang chờ trên semaphore s, bị khóa bởi thao tác Down, thì hệ thống sẽ chọn một trong các tiến trình này để kết thúc thao tác Down và cho tiếp tục xử lý.



***Hình 4.4.2.1-1**. Semaphore s*

Cài đặt: Gọi p là tiến trình thực hiện thao tác Down(s) hay Up(s).

Down(s):

```
e(s) = e(s) - 1;
if e(s) < 0 {
status(P)= blocked;
enter(P,f(s)); }
```

Up(s):

```
e(s) = e(s) + 1;
if s = 0 {
exit(Q,f(s)); //Q là tiến trình đang chờ trên s
status (Q) = ready;
enter(Q,ready-list); }
```

Lưu ý cài đặt này có thể đưa đến một giá trị âm cho semaphore, khi đó trị tuyệt đối của semaphore cho biết số tiến trình đang chờ trên semaphore. Điều quan trọng là các thao

tác này cần thực hiện một cách không bị phân chia, không bị ngắt giữa chừng, có nghĩa là không một tiến trình nào được phép truy xuất đến semaphore nếu tiến trình đang thao tác trên semaphore này chưa kết thúc xử lý hay chuyển sang trạng thái blocked.

Sử dụng: có thể dùng semaphore để giải quyết vấn đề truy xuất độc quyền hay tổ chức phối hợp giữa các tiến trình. Tổ chức truy xuất độc quyền với Semaphores: khái niệm semaphore cho phép bảo đảm nhiều tiến trình cùng truy xuất đến miền găng mà không có sự mâu thuẫn truy xuất. n tiến trình cùng sử dụng một semaphore s, e(s) được khởi gán là 1. Để thực hiện đồng bộ hóa, tất cả các tiến trình cần phải áp dụng cùng cấu trúc chương trình sau đây:

```
while (TRUE) {
```

Down(s)

```
critical-section ();
```

Up(s)

```
Noncritical-section (); }
```

Tổ chức đồng bộ hóa với Semaphores: với semaphore có thể đồng bộ hóa hoạt động của hai tiến trình trong tình huống một tiến trình phải đợi một tiến trình khác hoàn tất thao tác nào đó mới có thể bắt đầu hay tiếp tục xử lý. Hai tiến trình chia sẻ một semaphore s, khởi gán e(s) là 0. Cả hai tiến trình có cấu trúc như sau:

P1:

```
while (TRUE) {
job1();
Up(s); //đánh thức P2 }
```

P2:

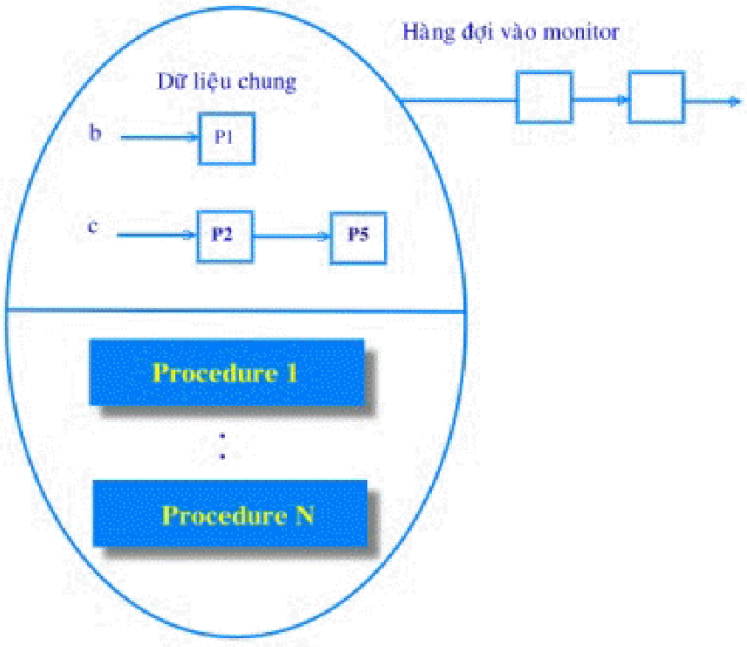
```
while (TRUE) {
Down(s); // chờ P1
job2(); }
```

Việc sử dụng đúng cách semaphore để đồng bộ hóa phụ thuộc hoàn toàn vào lập trình viên và đòi hỏi lập trình viên phải hết sức thận trọng.

4.4.2.2 Monitors

Tiếp cận: Để có thể dễ viết đúng các chương trình đồng bộ hóa hơn, Hoare(1974) và Brinch & Hansen (1975) đã đề nghị một cơ chế cao hơn được cung cấp bởi ngôn ngữ lập trình, là monitor. Monitor là một cấu trúc đặc biệt bao gồm các thủ tục, các biến và cấu trúc dữ liệu có các thuộc tính sau:

- Các biến và cấu trúc dữ liệu bên trong monitor chỉ có thể được thao tác bởi các thủ tục định nghĩa bên trong monitor đó (encapsulation).
- Tại một thời điểm, chỉ có một tiến trình duy nhất được hoạt động bên trong một monitor (mutual exclusive).
- Trong một monitor, có thể định nghĩa các biến điều kiện và hai thao tác kèm theo là Wait và Signal như sau: gọi c là biến điều kiện được định nghĩa trong monitor:
  - Wait(c): chuyển trạng thái tiến trình gọi sang blocked, và đặt tiến trình này vào hàng đợi trên biến điều kiện c.
  - Signal(c): nếu có một tiến trình đang bị khóa trong hàng đợi của c, tái kích hoạt tiến trình đó, và tiến trình gọi sẽ rời khỏi monitor.



***Hình 4.4.2.2-1**.Monitor và các biến điều kiện*

Cài đặt: trình biên dịch chịu trách nhiệm thực hiện việc truy xuất độc quyền đến dữ liệu trong monitor. Để thực hiện điều này, một semaphore nhị phân thường được sử dụng. Mỗi monitor có một hàng đợi toàn cục lưu các tiến trình đang chờ được vào monitor, ngoài ra, mỗi biến điều kiện c cũng gắn với một hàng đợi f(c) và hai thao tác trên đó được định nghĩa như sau:

Wait(c) :

```
status(P)= blocked;
enter(P,f(c));
```

Signal(c) :

```
if (f(c) != NULL){
exit(Q,f(c)); //Q là tiến trình chờ trên c
statusQ) = ready;
enter(Q,ready-list); }
```

Sử dụng: Với mỗi nhóm tài nguyên cần chia sẻ, có thể định nghĩa một monitor trong đó đặc tả tất cả các thao tác trên tài nguyên này với một số điều kiện nào đó:

```
monitor <tên monitor >
```

condition <danh sách các biến điều kiện>; <déclaration de variables>;

```
procedure Action1();
{
}
procedure Actionn();
{
}
end monitor;
```

Các tiến trình muốn sử dụng tài nguyên chung này chỉ có thể thao tác thông qua các thủ tục bên trong monitor được gắn kết với tài nguyên:

```
while (TRUE) {
Noncritical-section ();
<monitor>.Actioni; //critical-section();
Noncritical-section (); }
```

4.4.2.3 Trao đổi thông điệp

Tiếp cận: giải pháp này dựa trên cơ sở trao đổi thông điệp với hai primitive Send và Receive để thực hiện sự đồng bộ hóa:

- Send (destination, message): gởi một thông điệp đến một tiến trình hay gởi vào hộp thư.
- Receive (source,message): nhận một thông điệp thừ một tiến trình hay từ bất kỳ một tiến trình nào, tiến trình gọi sẽ chờ nếu không có thông điệp nào để nhận.

Sử dụng: Có nhiều cách thức để thực hiện việc truy xuất độc quyền bằng cơ chế trao đổi thông điệp. Đây là một mô hình đơn giản: một tiến trình kiểm soát việc sử dụng tài nguyên và nhiều tiến trình khác yêu cầu tài nguyên này. Tiến trình có yêu cầu tài nguyên sẽ gởi một thông điệp đến tiến trình kiểm soát và sau đó chuyển sang trạng thái blocked cho đến khi nhận được một thông điệp chấp nhận cho truy xuất từ tiến trình kiểm soát tài nguyên.Khi sử dụng xong tài nguyên, tiến trình gởi một thông điệp khác đến tiến trình kiểm soát để báo kết thúc truy xuất. Về phần tiến trình kiểm soát , khi nhận được thông điệp yêu cầu tài nguyên, nó sẽ chờ đến khi tài nguyên sẵn sàng để cấp phát thì gởi một thông điệp đến tiến trình đang bị khóa trên tài nguyên đó để đánh thức tiến trình này.

```
while (TRUE) {
Send(process controler, request message);
Receive(process controler, accept message);
critical-section ();
Send(process controler, end message);
Noncritical-section (); }
```

Câu hỏi ôn tập

1. Vì sao giải pháp dùng biến khóa (lock) không đảm bảo độc quyền truy xuất?
2. Giải thích vì sao giải pháp Peterson đảm bảo độc quyền truy xuất cho hai tiến trình.
3. Nhược điểm chung của các giải pháp busy waiting là gì?
4. Trình bày định nghĩa và cách cài đặt các thao tác Down(s), Up(s) của semaphore.
5. Monitor khác semaphore ở điểm nào? Vì sao monitor giúp giảm lỗi lập trình?

Học thêm, bài tập và hướng dẫn giải

Nguyên lí hệ điều hành — Bài tập mở rộng về tiến trình và đồng bộ

Tài liệu tự biên soạn bổ sung cho 11 bài hiện có, không thay thế bài giảng gốc. Phân biệt mã giả với API cụ thể; ví dụ về đồng bộ dùng mô hình lý thuyết.

Mục tiêu học thêm

- Phân biệt đồng thời, song song, tiến trình và tiểu trình
- Phân tích trạng thái tiến trình và lựa chọn cơ chế liên lạc
- Tìm race condition, deadlock và lỗi thứ tự lấy khóa

1. Đồng thời và song song

Đồng thời nghĩa là nhiều công việc có tiến triển trong các khoảng thời gian chồng lấn; trên một lõi, hệ điều hành có thể xen kẽ chúng. Song song là thực thi tại cùng thời điểm trên nhiều đơn vị xử lý. Có nhiều thread không bảo đảm chương trình nhanh hơn: còn phụ thuộc khối lượng công việc, chờ I/O, tranh chấp khóa và chi phí chuyển ngữ cảnh.

Tiến trình có không gian địa chỉ và tài nguyên riêng theo mô hình hệ điều hành. Các thread cùng tiến trình thường chia sẻ không gian địa chỉ nhưng mỗi thread có stack và trạng thái thực thi riêng. Chia sẻ dữ liệu thuận tiện hơn, đồng thời cũng dễ phát sinh tranh chấp.

2. Trạng thái và lập lịch

Một mô hình đơn giản có ready, running và blocked. Ready là đủ điều kiện chạy nhưng chờ CPU; blocked là đang chờ sự kiện như I/O. Khi hết quantum, tiến trình running có thể quay về ready. Hoàn tất I/O thường đưa blocked về ready, không nhất thiết được chạy ngay.

Ví dụ Round Robin: A cần 5 đơn vị CPU, B cần 3, cùng tới thời điểm 0, quantum=2, hàng đợi ban đầu A rồi B, không có I/O và bỏ qua chi phí chuyển ngữ cảnh.

- A chạy 0–2, còn 3.
- B chạy 2–4, còn 1.
- A chạy 4–6, còn 1.
- B chạy 6–7, hoàn tất.
- A chạy 7–8, hoàn tất.

Turnaround của A là 8, B là 7. Waiting time bằng turnaround trừ tổng CPU trong mô hình này: A=3, B=4. Nếu có I/O thì không áp dụng máy móc công thức trừ CPU mà bỏ qua thời gian bị blocked.

3. Lựa chọn IPC

Pipe phù hợp luồng dữ liệu giữa các tiến trình trong tình huống thích hợp trên cùng máy; socket phù hợp liên lạc qua mạng hoặc cùng máy. Shared memory cho phép chia sẻ vùng dữ liệu nhưng cần tự giải quyết đồng bộ và quyền truy cập. Message passing giúp tách vùng nhớ, nhưng phải xem xét kích thước thông điệp, sao chép và việc gửi/nhận có chặn hay không.

Không chọn IPC chỉ vì "nhanh nhất". Cần biết các tiến trình có cùng máy không, có quan hệ cha/con không, dữ liệu lớn bao nhiêu, có bao nhiêu bên gửi/nhận và cần xử lý lỗi bên kia dừng thế nào.

4. Race condition

Phép counter = counter + 1 có thể gồm đọc, tính và ghi, không nên mặc định nguyên tử. Hai thread cùng đọc 0, cùng tính 1 rồi cùng ghi 1 làm mất một lần tăng; kết quả đúng mong đợi là 2.

Mutex bảo vệ vùng đọc-sửa-ghi; mọi đường truy cập dữ liệu chia sẻ cần tuân thủ cùng quy tắc khóa. Không chỉ khóa dòng ghi cuối vì hai thread vẫn có thể tính từ dữ liệu cũ. Cần bảo đảm khóa được giải phóng khi có lỗi.

5. Producer–consumer và thứ tự semaphore

Với buffer có N ô, đặt empty=N, full=0 và mutex=1. wait giảm semaphore hoặc chờ khi không thể giảm; signal tăng và có thể đánh thức bên chờ.

```
Producer:
  wait(empty)
  wait(mutex)
  thêm dữ liệu vào buffer
  signal(mutex)
  signal(full)

Consumer:
  wait(full)
  wait(mutex)
  lấy dữ liệu khỏi buffer
  signal(mutex)
  signal(empty)
```

Không lấy mutex rồi mới chờ empty/full: nếu giữ mutex trong khi buffer đầy/rỗng, bên cần thay đổi buffer có thể không lấy được mutex để giải phóng điều kiện chờ. Với condition variable, thường kiểm tra điều kiện trong vòng while dưới mutex và dùng thao tác chờ phù hợp giải phóng mutex; không dùng if một lần rồi giả định điều kiện luôn còn đúng khi thức dậy.

6. Deadlock và starvation

Deadlock là các bên không thể tiếp tục do chờ tài nguyên nhau. Ví dụ T1 giữ A chờ B, T2 giữ B chờ A. Quy định mọi thread lấy A trước B loại vòng chờ này. Starvation là một bên liên tục không được phục vụ dù hệ thống vẫn tiến triển; không đồng nghĩa với deadlock. Một khóa có mutual exclusion chưa tự bảo đảm fairness.

Bài tập và gợi ý đáp án

1. Dễ: I/O hoàn tất có nghĩa tiến trình chạy ngay không? Không: thông thường chuyển về ready rồi chờ lập lịch.
2. Trung bình: lập bảng Round Robin ở ví dụ và tính thời điểm phản hồi đầu tiên. A=0, B=2; kiểm tra tổng CPU là 8.
3. Khó: producer giữ mutex rồi chờ empty khi buffer đầy. Consumer cần mutex để lấy phần tử nên không giải phóng empty được; đổi thứ tự như mã giả trên. Phân tích thêm lỗi nếu quên signal(mutex).

Tiêu chí và nguồn học tiếp

Vẽ timeline xen kẽ từng thao tác, ghi trạng thái biến và giá trị semaphore; không chỉ kết luận "có lỗi". Với lời giải đồng bộ, chỉ ra vùng tới hạn, điều kiện chờ và bên có thể làm điều kiện thay đổi. Với lập lịch, luôn ghi giả định về thời điểm đến, quantum và chi phí chuyển ngữ cảnh.

https://pages.cs.wisc.edu/~remzi/OSTEP/ ; các chương CPU Scheduling, Threads, Locks, Condition Variables và Semaphores.

Bài 11: Các giải pháp đồng bộ hóa — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: dùng lock nhưng vẫn treo vì thứ tự lấy nhiều khóa không nhất quán. Quy định cùng một thứ tự lấy khóa và giải phóng ở mọi đường thoát. Semaphore đếm tài nguyên; mutex bảo vệ vùng truy cập theo hợp đồng API, không dùng hai khái niệm như đồng nghĩa tuyệt đối.

Tự luyện: mô hình hàng đợi hữu hạn producer/consumer: giới hạn sức chứa, số phần tử và chỗ trống. **Hướng giải:** kiểm tra bất biến 0 ≤ size ≤ capacity, không giữ khóa lâu trong I/O nếu không cần. Với condition variable, kiểm tra lại điều kiện sau khi thức dậy. Muốn tìm ứng dụng nên đóng trên máy lag phải có dữ liệu thực; không đoán tên chương trình từ tài liệu. Bộ nhớ ảo mở rộng mô hình không gian địa chỉ nhưng paging ra đĩa không làm RAM vật lý nhanh hơn.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/